

Agentic ESOpt: Fine-Tuning Long-Horizon LLM Agents with Minimal GPU Requirements

Zhi Zheng¹, Rongsheng Chen², Yunpeng Ba², Zhenkun Wang², Yee Whye Teh³, Wee Sun Lee¹

¹National University of Singapore, ²Southern University of Science and Technology, ³Oxford

Abstract

Reinforcement Learning (RL) has been promising in single-turn LLM fine-tuning. However, long-horizon agentic reasoning introduces increasingly branching interactions and sparse rewards, exposing several limitations of RL: its heavyweight backpropagation makes it impractical to fine-tune larger LLMs, and longer-horizon trajectories make the credit assignment substantially harder. This paper argues that evolution strategies (ES) can be a better choice for fine-tuning long-horizon LLM agents. Compared with agentic RL, ES offers three key advantages: **1) Model Scalability:** ES enables full-parameter optimization requiring only minimal, inference-level GPU memory, making it possible to fine-tune large LLMs. **2) Flexibility:** its lightweight, black-box feedback interface makes ES fine-tuning easy to compose with prompt-space evolution (e.g., skill optimization & test-time compute); and **3) Long-Horizon Scalability:** ES performs trajectory-level parameter attribution without decomposing rewards across horizons, yielding better scalability than Agentic RL as the horizon length grows.

Based on this insight, we propose **Agentic ESOpt**, a full-parameter agentic fine-tuning framework tailored to flexible parameter-context co-evolution. At each step, Agentic ESOpt samples perturbations around the current LLM parameters, evaluates the resulting agents with rewards, and applies an online reward-weighted update. To improve the exploration-adaptation trade-off, Agentic ESOpt further introduces a cosine decay schedule of the perturbation scale σ . We evaluate Agentic ESOpt across both train-time fine-tuning and agentic test-time compute settings. On long-horizon Sudoku, Agentic ESOpt outperforms RL methods by 12.50% with Qwen3.5-4B. On WebArena-Lite, full-parameter optimization of **Qwen3.5-27B** improves the No Skill baseline by 6.69%, and combining Agentic ESOpt with Trace2Skill further improves the Trace2Skill baseline by 2.42%. In test-time automatic heuristic design, Agentic ESOpt performs online prompt-parameter co-evolution, improving its matched baseline in 28 of 36 settings.

GitHub : <https://github.com/zz1358m/Agentic-ESOpt>

Correspondence: zhi.zheng@u.nus.edu

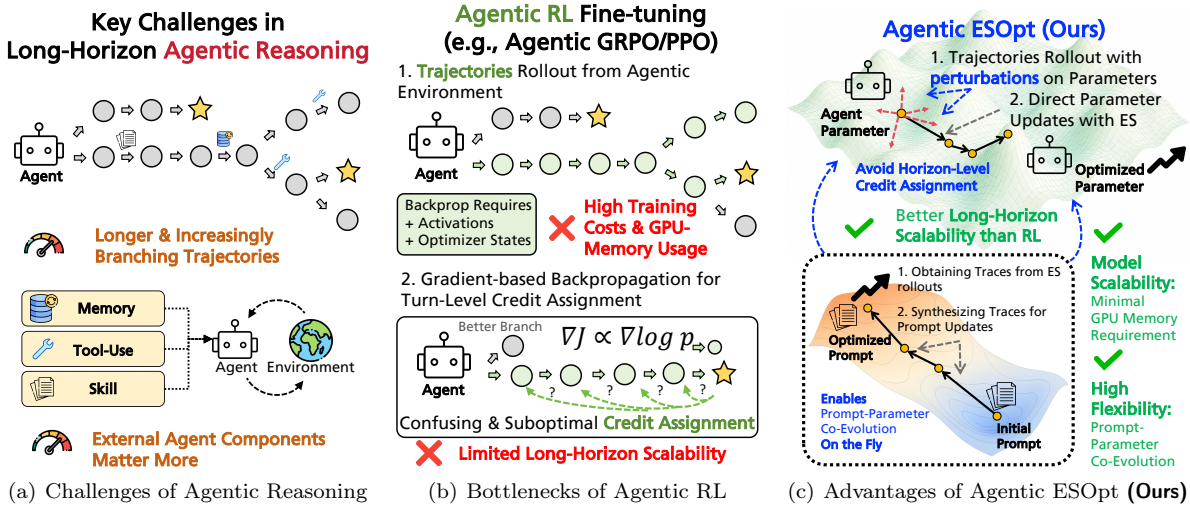


Figure 1 (a) Long-horizon agentic reasoning introduces new challenges for Agentic RL (b), including high GPU memory requirements in training and difficult credit assignment across horizons. Agentic ESOpt (c) addresses these issues through inference-level GPU memory, flexible black-box feedback, and better long-horizon scalability.

1 Introduction

Advanced Large Language Models (LLMs) such as Qwen3, DeepSeek-R1, and Gemini 2.5 have demonstrated strong capabilities as general-purpose agents (Yang et al., 2025; Guo et al., 2025; Gemini Team, 2025). With their capabilities in tool use, long-context processing, and multimodal interaction, these models can navigate websites (Zhou et al., 2024), edit repositories (Yang et al., 2024), and coordinate multi-step software workflows (Wang et al., 2024). However, general-purpose agents can still perform poorly on uncommon tool APIs (Ma et al., 2024) and specialized scientific or algorithmic tasks (Liu et al., 2024). Therefore, efficiently fine-tuning advanced LLM agents to adapt task-specific expertise remains important (Du et al., 2026).

Reinforcement learning (RL) has demonstrated remarkable effectiveness in single-turn LLM fine-tuning (Shao et al., 2024; Liu et al., 2025d; Zheng et al., 2025a; Tajwar et al., 2026). However, in long-horizon agentic reasoning, which introduces increasingly branching interactions and provides only sparse feedback, several limitations of Agentic RL are exposed. As illustrated in Figure 1, first, Agentic RL requires storing heavyweight activations, optimizer states, and performing backpropagation through trajectories, making full-parameter fine-tuning increasingly impractical for larger LLMs. Moreover, as trajectories become longer and more branching, assigning sparse trajectory-level rewards back to individual decisions becomes substantially harder (Kim et al., 2026).

This paper argues that **evolution strategies (ES)** (Salimans et al., 2017) can be a better choice for fine-tuning long-horizon LLM agents. Instead of performing backpropagation, ES samples perturbations around the current LLM parameters, evaluates the perturbed agents with environment rewards, and applies a reward-weighted parameter update. Compared with Agentic RL, ES offers three key advantages:

- **1) Model Scalability:** ES enables full-parameter optimization with only inference-level GPU memory, which is the minimal amount, substantially reducing the memory barrier to fine-tuning larger LLM agents.
- **2) Flexibility:** Its lightweight, black-box feedback interface makes ES fine-tuning easy to compose with skill-space evolution (Ni et al., 2026) and test-time compute (Liu et al., 2024).
- **3) Long-Horizon Scalability:** Unlike RL estimators that usually lead to poor long-horizon credit assignment, ES performs trajectory-level parameter attribution without decomposing rewards across turns, yielding better scalability than Agentic RL as the horizon length grows.

Recent work has explored ES for single-turn LLM reasoning (Qiu et al., 2026; Sun et al., 2026; Sarkar et al., 2025), where ES achieves higher GPU memory efficiency but slightly lower performance than RL methods. However, we argue that the structural advantages of ES are **particularly pronounced in fine-tuning long-horizon agents**, where ES can be significantly preferable to RL, rather than merely a cheaper alternative. To implement this advantage, we propose **Agentic ESOpt**, a full-parameter ES framework for both train-time agent fine-tuning and agentic test-time compute. At each generation, Agentic ESOpt samples full-parameter perturbations, evaluates the resulting agents with environment rewards, and applies an online reward-weighted update. Its lightweight black-box update enables on-the-fly parameter adaptation within prompt-space optimization loops, allowing parameter updates to complement both skill-space optimization and test-time search. To improve the exploration–adaptation trade-off, Agentic ESOpt further introduces a cosine decay mechanism for the perturbation scale σ , which retains a nonzero terminal σ_T for mild smoothing regularization in train-time optimization, and decays σ_T to 0 for progressively finer adaptation in test-time optimization.

We evaluate **Agentic ESOpt** for agentic fine-tuning in both train-time and test-time compute with LLMs from 4B to 27B. In the **train-time fine-tuning**, we study long-horizon reasoning, ReAct-style tool use, and web agents. On Sudoku, RL- and ES-based methods with matched FLOPs are competitive at smaller minimum successful horizons (5- and 10-horizon), but their relative ordering changes as the horizon grows: at 15 turns, Agentic ESOpt reaches +12.5% compared to the strongest GRPO baseline. Across ReAct-style Math and DocVQA, Agentic ESOpt achieves an average improvement of 13.7% over the *Qwen3.5-4B* base model and 8.3% over Agentic GRPO. On WebArena-Lite, full-parameter optimization of *Qwen3.5-27B* improves the No Skill baseline from 29.47% to 36.16%, while combining Agentic ESOpt with Trace2Skill raises it from 33.94% to 36.36%. In the **test-time** agentic heuristic design, Agentic ESOpt improves the matched baselines in 28 of 36 comparisons. Moreover, a preliminary population-sensitivity study further suggests that stronger LLM backbones can obtain useful ES updates with smaller populations. Our contributions are summarized as follows:

- We identify that, on long-horizon agentic reasoning, ES becomes preferable to Agentic RL. We attribute this shift to three key properties of ES: **model scalability** through inference-level GPU memory, **flexibility** through black-box trajectory feedback, and **long-horizon scalability** through trajectory-level parameter attribution.

- We introduce **Agentic ESOpt**, a backpropagation-free ES framework require only minimal GPU memory. It supports flexible agentic fine-tuning in both **train-time adaptation** and **test-time compute**. It allows parameter optimization to compose with prompt-space evolution, while a cosine schedule of perturbation improves the exploration–adaptation trade-off.
- We validate **Agentic ESOpt** across long-horizon Sudoku, ReAct-style tool use, web agents, and automatic heuristic design with models from 4B to 27B. Agentic ESOpt outperforms RL on long-horizon Sudoku and ReAct-style Math/DocVQA, enables full-parameter adaptation of a 27B WebArena agent, improves Trace2Skill, and enhances existing test-time evolutionary search in 28 of 36 settings.

2 Preliminaries: Agentic LLM Reasoning

We define a multi-turn LLM agent that repeatedly observes the environment and produces an action as follows:

$$a_t \sim \pi_\theta(a_t \mid \mathbf{o}_{\leq t}, c_t),$$

where θ denotes model parameters, $\mathbf{o}_{\leq t}$ is the interaction history, and c_t is an external prompt, memory, skill, or tool instruction. An episode induces a multi-turn trajectory $\tau = (o_1, a_1, \dots, o_H, a_H)$, where the **horizon** H is the number of agent–environment turns before termination. Its return is $R(\tau) = \sum_{t=1}^H \gamma^{t-1} r_t$. In many agentic reasoning tasks, rewards are sparse: intermediate rewards are typically zero, $r_t = 0$ for $t < H$, and only the completed trajectory receives a task score. In some problems (e.g., MATH & DocVQA ReAct-style Tool Usage), the agent receives the complete multi-turn trajectory τ as input, but in some problems with (partial) Markov properties (Sudoku, WebArena), the agent only needs to receive partial local input (e.g., o_t only).

The optimization of the Task-specific agent can act on c_t or θ . Prompt-space optimization methods optimize c_t while freezing the LLM. They are lightweight, but can only elicit behaviors already accessible to the frozen policy (Yang et al., 2026; Ni et al., 2026; Liu et al., 2024; Zheng et al., 2025b). Policy fine-tuning methods optimize θ for better agentic ability with Supervised Fine-Tuning (SFT), On-Policy Distillation (OPD), Group Relative Policy Optimization (GRPO), or Proximal Policy Optimization (PPO).

Agentic SFT & OPD. Agentic SFT learns from expert actions (Chu et al., 2025), while Agentic OPD learns from token distributions of higher-ability LLMs (Song and Zheng, 2026). Both methods require labels beyond the scalar environment reward, which requires additional cost, so we exclude them from our comparison scope.

Agentic PPO & GRPO. Agentic PPO and Agentic GRPO require a scalar environment reward, making them useful in many application scenarios (Qi et al., 2025). Agentic GRPO (Shao et al., 2024) samples G trajectories for the same task and assigns trajectory i the group-relative advantage as follows:

$$\hat{A}_i = \frac{R_i - \text{mean}_j(R_j)}{\text{std}_j(R_j) + \varepsilon}.$$

Let $q_{i,t}(\theta) = \pi_\theta(a_{i,t} \mid h_{i,t}) / \pi_{\theta_{\text{old}}}(a_{i,t} \mid h_{i,t})$, where $h_{i,t} = (\mathbf{o}_{\leq t}, c_t)$. Agentic GRPO minimizes the loss as follows:

$$\mathcal{L}_{\text{GRPO}} = -\frac{1}{G} \sum_{i=1}^G \sum_{t=1}^{H_i} \min\left(q_{i,t} \hat{A}_i, \text{clip}(q_{i,t}, 1 - \epsilon, 1 + \epsilon) \hat{A}_i\right) + \beta D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}}), \quad (1)$$

However, recent studies (Feng et al., 2026; He et al., 2026; Zhang, 2026) point out that although GRPO’s group-relative advantage is representative in single-turn scenarios, it cannot cover multi-turn scenarios. To achieve better credit assignment, PPO has recently been adopted in multi-turn agentic RL (Li et al., 2026; Hou et al., 2026), which uses the same clipped surrogate but replaces $\hat{A}_i$ with a turn-level advantage estimated by a critic LLM (Schulman et al., 2017):

$$\mathcal{L}_{\text{PPO}} = -\sum_{t=1}^H \min\left(q_t \hat{A}_t, \text{clip}(q_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t\right), \quad \hat{A}_t = \sum_{l=0}^{H-t} (\gamma \lambda)^l (r_{t+l} + \gamma V_\phi(h_{t+l+1}) - V_\phi(h_{t+l})). \quad (2)$$

We argue that PPO cannot fully remove the long-horizon difficulty. It requires a critic warm-up phase to learn meaningful values, while sparse terminal rewards make its early advantages unreliable. Even with a well-trained critic, the policy gradient still sums H action-level score terms, so its variance remains dependent on the horizon length. These limitations motivate the trajectory-level estimator introduced next and analyzed in Appendix C.3. These drawbacks in credit assignment motivate the parameter-space ES formulation of **Agentic ESOpt**.

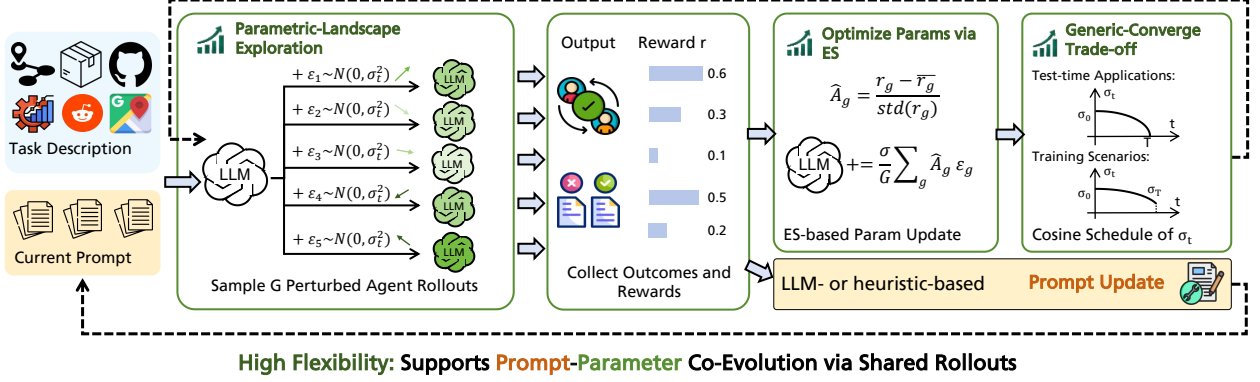


Figure 2 Detailed workflow of Agentic ESOpt. Starting from the current LLM, Agentic ESOpt samples parameter perturbations, evaluates the perturbed agents in the environment, normalizes their scalar rewards, and applies a reward-weighted ES update. Compared with Agentic RL, Agentic ESOpt provides **model scalability**, **optimization flexibility**, and **long-horizon scalability**. Its lightweight black-box interface also allows easy composition with prompt-space optimization methods such as Trace2Skill (LLM-based) and EoH (heuristic-based), enabling on-the-fly parameter adaptation within existing test-time compute procedures.

3 Methodology: Agentic ESOpt

As illustrated in Figure 2, **Agentic ESOpt** performs full-parameter ES optimization by sampling parameter perturbations around the current LLM, evaluating the perturbed agents with scalar environment rewards, and applying a reward-weighted parameter update. This update is forward-only, which only requires storing the noise seed and using in-place addition and subtraction (Qiu et al., 2026), so the GPU memory requirement of Agentic ESOpt remains minimal, the same as the inference requirements. Moreover, the same black-box trajectory feedback can be reused by skill-space optimizers or test-time compute.

Formally, let $\tau = (o_0, a_0, \dots, o_H, a_H)$ denote an interaction trajectory induced by policy π_θ , and let $R(\tau)$ denote its scalar trajectory return. For a fixed external agent state c , the objective is as follows:

$$J(\theta; c) = \mathbb{E}_{\tau \sim \pi_\theta(\cdot|c)} [R(\tau)].$$

Agentic ESOpt optimizes the objective by searching over the parameter space around θ , where $\epsilon \in \mathbb{R}^d$ is a full-parameter perturbation on the d -dimensional parameters, and the Gaussian-smoothed objective is as:

$$J_\sigma(\theta; c) = \mathbb{E}_{\epsilon \sim \mathcal{N}(0, I)} [J(\theta + \sigma\epsilon; c)]. \quad (3)$$

Then we can derive the ES pseudo-gradient as follows (Salimans et al., 2017; Qiu et al., 2026):

$$\nabla_\theta J_\sigma(\theta; c) = \frac{1}{\sigma} \mathbb{E}_\epsilon [J(\theta + \sigma\epsilon; c)\epsilon]. \quad (4)$$

The derivation is detailed in Appendix C.1. The ES gradients are estimated from scalar scores without differentiating through the agent-environment interaction.

In implementation, to estimate Eq.(4), Agentic ESOpt samples G perturbations $\epsilon_1, \dots, \epsilon_G$, evaluates the corresponding perturbed agents, and obtains rewards $R_i = R(\tau_i)$. To reduce variance, we normalize the rewards within the population with a z-score as follows:

$$\hat{R}_i = \frac{R_i - \mu_R}{s_R + \varepsilon}, \quad \text{where} \quad \mu_R = \frac{1}{G} \sum_{j=1}^G R_j, \quad s_R^2 = \frac{1}{G} \sum_{j=1}^G (R_j - \mu_R)^2.$$

In practice, unlike the canonical ES estimator in Eq. (4), our normalized update omits the explicit $1/\sigma$ factor, with α serving as the effective update scale. The implemented update is therefore:

$$\theta_{t+1} = \theta_t + \frac{\alpha}{G} \sum_{i=1}^G \hat{R}_i \epsilon_i.$$

We follow Qiu et al. (2026) in storing only the noise seed of each perturbation and using in-place addition and subtraction for each perturbed agent, so Agentic ESOpt requires only the same amount of GPU memory for inference.

Prompt-Space Composition and Prompt-Parameter Co-Evolution. Test-time compute and prompt-space optimization methods typically keep the LLM parameters fixed throughout the search process. As a result, the search can only reweight or recombine behaviors already accessible to the frozen policy, which may limit optimization when solving the task requires changing the underlying policy itself. In contrast, the lightweight black-box updates of **Agentic ESOpt** allow parameter adaptation to be performed on the fly alongside prompt-space search, **enabling prompt-parameter co-evolution**.

Let $\mathcal{D}_t$ denote the trajectories and scores collected at iteration t , $\mathcal{U}_{\text{ES}}$ the Agentic ESOpt parameter update, and $\mathcal{U}_c$ an external update rule for prompt c_t . A general alternating outer loop can then update the two spaces as

$$\theta_{t+1} = \mathcal{U}_{\text{ES}}(\theta_t; c_t, \mathcal{D}_t), \quad c_{t+1} = \mathcal{U}_c(c_t; \mathcal{D}_t).$$

3.1 Cosine Decay of the Perturbation Radius σ

With perturbations $\epsilon \sim \mathcal{N}(0, I)$, Agentic ESOpt effectively optimizes the Gaussian-smoothed objective in Eq (3). However, a nonzero perturbation radius introduces a smoothing bias, whose leading term is characterized as:

Lemma 1 (Gaussian-smoothing bias). *Assume $J(\theta; c)$ is sufficiently smooth in a neighborhood of θ . Then*

$$J_\sigma(\theta; c) = J(\theta; c) + \frac{\sigma^2}{2} \text{Tr}(\nabla_\theta^2 J(\theta; c)) + O(\sigma^4).$$

The proof is in Appendix C.2. The second-order term $\text{Tr}(\nabla_\theta^2 J)$ can be viewed as a **regularization term**. For a maximization objective, it penalizes sharp local optima while favoring flatter parameter neighborhoods. Therefore, a larger σ introduces stronger regularization but also a larger bias from the original objective.

Existing ES methods for single-turn LLM fine-tuning typically use a fixed perturbation radius throughout optimization (Qiu et al., 2026), without explicitly adapting this trade-off. Agentic ESOpt instead gradually decreases σ over T update steps, using a larger radius early for broader exploration and stronger regularization, and a smaller radius later for increased exploitation and reduced objective bias as follows:

$$\sigma_t = \sigma_T + (\sigma_0 - \sigma_T) \frac{1 + \cos(\pi t/T)}{2}, \quad t = 0, \dots, T.$$

For **train-time Agentic ESOpt**, we retain a nonzero σ_T to balance exploitation with exploration and regularization. In contrast, **test-time compute** focuses on the unbiased outcome of the current task instead of the generalization of the agent, so we decay σ_T to zero to minimize the objective bias toward the end of optimization.

4 Agentic Sudoku: Controlled Experiments on Long-Horizon Scalability

To demonstrate the advantage of Agentic ESOpt on **long-horizon scalability** compared to Agentic RL, this section first analyzes their characteristics theoretically. Then we evaluate the proposed Agentic ESOpt in a controlled multi-turn Sudoku environment, where the minimal possible horizon length H is strictly defined by the number of masks on the Sudoku board.

Theoretical Reason for Scalability. Consider a trajectory of H actions, $\mathbf{a} = (a_1, \dots, a_H)$, with terminal return $R(\mathbf{a})$. A simple Agentic RL estimator (e.g., Eq (1), Eq (2)) with baseline b has the form as follows:

$$\hat{g}_{\text{PG}} = (R(\mathbf{a}) - b) \sum_{t=1}^H \nabla_\theta \log \pi_\theta(a_t | s_t).$$

Following the analysis of Salimans et al. (2017, Sec. 3.1), suppose the return has weak correlation with any individual action, the per-step score terms are approximately uncorrelated, and ES and policy gradients induce comparable return variation. The estimator variance will then grow nearly linearly with the horizon as follows:

$$\text{Var}[\hat{g}_{\text{PG}}] \approx \text{Var}[R(\mathbf{a})] \text{Var}\left[\sum_{t=1}^H \nabla_\theta \log \pi_\theta(a_t | s_t)\right] \propto H.$$

Agentic ESOpt samples one parameter perturbation ϵ for the complete rollout. Its corresponding estimator is as follows:

$$\hat{g}_{\text{ES}} = (R(a(\theta + \sigma\epsilon)) - b) \frac{\epsilon}{\sigma}, \quad \text{Var}[\hat{g}_{\text{ES}}] \approx \text{Var}[R(\mathbf{a})] \text{Var}\left[\frac{\epsilon}{\sigma}\right].$$

So, the parameter-score term ϵ/σ does not sum over H . Agentic ESOpt therefore assigns the terminal return directly to one coherent policy variation, without asking the same scalar outcome to distinguish among H horizons. This scaling argument predicts a **relative** advantage for Agentic ESOpt as the effective horizon grows.

Importantly, this comparison isolates the horizon-dependent structure of the two estimators. Agentic RL accumulates action-score terms across turns, whereas Agentic ESOpt avoids this with direct parameter-space search whose variance does not explicitly grow with H . This predicts an increasing relative advantage for parameter-space attribution as the effective horizon grows. A complete covariance expansion and discussion of other sources of estimator difficulty are provided in Appendix C.3. Figure 5 further visualizes local parameter-space neighborhoods as H^* increases. Although the reward contrast decreases for harder settings, the ES parameter score itself does not introduce an additional sum over turns.

Controlled Long-Horizon Sudoku. To empirically test the predicted long-horizon scaling behavior, we design a multi-turn Sudoku environment with a controllable **minimum successful horizon**. The environment provides **only a terminal reward**, and each valid action fills at most one cell. We define the shortest successful horizon for task x as $H^*(x) = \min_{\tau: R(\tau)=1} |\tau|$. Masking 5, 10, or 15 cells therefore gives $H^* \in \{5, 10, 15\}$. The realized horizon $H = |\tau|$ may exceed H^* because of invalid or unproductive actions. Our theoretical analysis concerns the realized horizon H , while the experiments are grouped by H^* .

Experimental Setup. Sudoku experiments are conducted on 4×NVIDIA H100 80GB GPUs. We compare Agentic ESOpt with two 8-rollout Agentic GRPO runs with different sampling configurations, Agentic PPO (Li et al., 2026), and Vanilla Agentic ES on Qwen3.5-4B. We create a 32-instance training dataset and a 32-instance evaluation dataset for each H^* . Following the recommendation in the model card, the temperature is 0.7, top-p is 0.8, and top-k is 20 across evaluations. We use task success rate as the primary metric, where an episode is successful only if the Sudoku is fully solved within the interaction budget, and report the mean and standard deviation over three evaluation runs. Detailed training and decoding configurations are in the Appendix D.1.1.

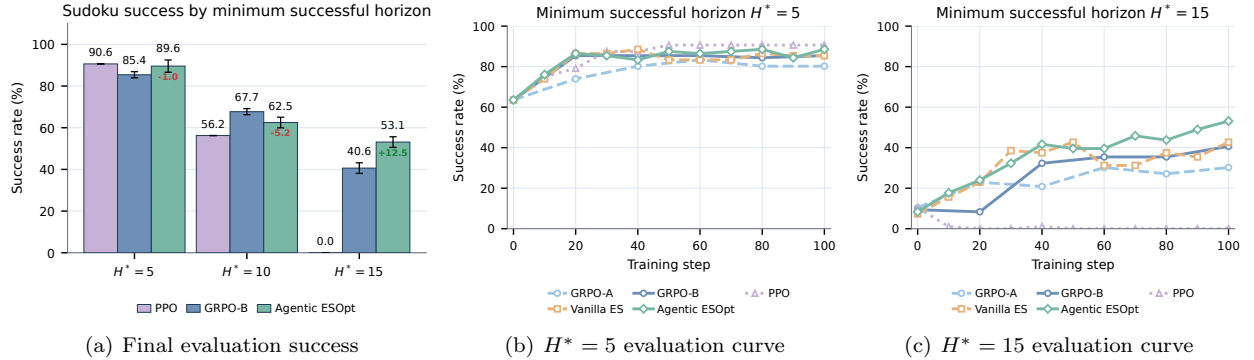


Figure 3 Agentic Sudoku performance grouped by minimum successful horizon H^* . (a) reports final success rate averaged over 3 runs with standard-deviation error bars for PPO, the stronger GRPO-B configuration, and Agentic ESOpt. Red/green annotations below the Agentic ESOpt values report its difference from the stronger Agentic RL result. (b) and (c) show evaluation curves for the methods. Both Vanilla ES and Agentic ESOpt use $G = 32$.

Table 1 Agentic Sudoku final evaluation success rate ($\times 100$) and GPU memory requirement, grouped by minimum successful horizon H^* . Values are reported as mean $\pm$ standard deviation.

Method	GPU Mem Req.	$H^* = 5$	$H^* = 10$	$H^* = 15$
Qwen3.5-27B	51.75GB	86.46 $\pm$ 3.90	50.00 $\pm$ 2.55	28.13 $\pm$ 2.55
Qwen3.5-4B	8.41GB	63.54 $\pm$ 7.80	31.25 $\pm$ 4.42	10.42 $\pm$ 1.47
+ Agentic PPO*	89.40GB	90.63 $\pm$ 0.00	56.25 $\pm$ 0.00	0.00 $\pm$ 0.00
+ Agentic GRPO [†]	58.88GB	80.21 $\pm$ 1.47	44.79 $\pm$ 2.95	30.21 $\pm$ 2.95
+ Agentic GRPO [‡]	58.88GB	85.42 $\pm$ 1.47	67.71 $\pm$ 1.47	40.63 $\pm$ 2.55
+ Agentic ESOpt (G=32)	8.41GB	89.58 $\pm$ 2.95	62.50 $\pm$ 2.55	53.13 $\pm$ 2.55
w/o σ decay (Vanilla ES)	8.41GB	85.42 $\pm$ 3.90	55.21 $\pm$ 5.89	42.71 $\pm$ 3.90
w/o σ_T (i.e., $\sigma_t = 0$)	8.41GB	85.42 $\pm$ 3.90	54.17 $\pm$ 3.90	28.13 $\pm$ 2.55

*PPO training use temperature 1, top- $p = 1$, and top- $k = -1$. [†]GRPO training uses temperature 0.7, top- $p = 0.8$, and top- $k = 20$. [‡]GRPO training uses temperature 1, top- $p = 1$, and top- $k = -1$. All GRPO and PPO evaluations, as well as Vanilla ES and Agentic ESOpt training and evaluation rollouts, use the recommended temperature-0.7, top- $p = 0.8$, top- $k = 20$.

Table 1 and Figure 3 (a) show a clear tendency that changes with the minimum successful horizon. At $H^* = 5$, PPO leads with 90.63%, followed by Agentic ESOpt at 89.58% and the stronger GRPO configuration at 85.42%. At $H^* = 10$, GRPO leads with 67.71%, followed by Agentic ESOpt at 62.50% and PPO at 56.25%. At $H^* = 15$, Agentic ESOpt becomes strongest at 53.13%, 12.50 percentage points above GRPO at 40.63%, while PPO becomes ineffective with only terminal reward. Under sparse terminal rewards, its critic fails to learn a reliable value signal, making the resulting advantage estimates uninformative for credit assignment. Agentic ESOpt does not dominate with small H^* ; its relative advantage appears as H^* grows, where the credit assignment of Agentic RL becomes harder. The ordering reversal—PPO at $H^* = 5$, GRPO at $H^* = 10$, and Agentic ESOpt at $H^* = 15$ —is therefore more informative than a uniform win: it is consistent with a horizon-dependent advantage regime rather than a globally stronger optimizer or a single favorable decoding configuration.

The evaluation curves Figure 3 (b,c) clarify how this separation develops. At $H^* = 5$, methods have a similar learning curve. However, at $H^* = 15$, GRPO variants fail to make fast improvements in the initial steps of training, while PPO quickly collapses. Their performance is related to the poor long-horizon credit assignment, where Agentic RL usually reaches the limit of the horizon (set to 45 for $H^* = 15$). In Figure 6 (b), the $H^* = 15$ setting has a 45-turn interaction budget; the GRPO turn count improves at around step 60, whereas Agentic ESOpt stays close to the minimum successful horizon and ends at 15.41 turns.

Compared to Vanilla ES, Agentic ESOpt can improve late in the longer run, from 39.58% at step 60 to 53.13% at step 100. This means that earlier, more extensive exploration given by the sigma schedule avoids the algorithm getting trapped in local optima. As shown in Table 1, removing cosine decay over the sigma in Vanilla Agentic ES leads to worse results, while setting the final sigma to 0 results in overfitting and poor evaluation performance.

Takeaway 1

Under the *weak-correlation scaling assumptions* used in the OpenAI ES analysis (Salimans et al., 2017, Sec. 3.1), parameter-space ES avoids the explicit horizon-wise score accumulation of action-space policy gradients. The Sudoku experiment exhibits the predicted **horizon-dependent crossover**: Agentic ESOpt is not uniformly strongest at short horizons, but becomes strongest at the largest controlled H^* .

Table 2 Training compute and wall-clock time on agentic Sudoku. Both Agentic GRPO and Agentic ESOpt are executed on the same four NVIDIA H100 GPUs and fully utilize the available hardware.

Method	$H^* = 5$		$H^* = 10$		$H^* = 15$	
	FLOPs	Time	FLOPs	Time	FLOPs	Time
Qwen3.5-4B + Agentic GRPO	3.2 EFLOPs	5.4 h	7.6 EFLOPs	13.1 h	10.9 EFLOPs	19.0 h
Qwen3.5-4B + Agentic ESOpt ($G=32$)	3.1 EFLOPs	3.1 h	6.3 EFLOPs	5.8 h	9.4 EFLOPs	9.4 h

Compute and wall-clock efficiency. The memory and compute results characterize two complementary efficiency properties of Agentic ESOpt. The GPU-memory results in Table 1 measure its minimal training-side requirement: Agentic ESOpt requires only 8.41GB, equal to the inference memory of the Qwen3.5-4B backbone and 85.7% below GRPO’s 58.88GB requirement.

ES requires a larger population ($G = 32$ versus 8-rollout in GRPO); however, it does not introduce a corresponding model-compute disadvantage. As analyzed in Appendix C.5, Agentic ESOpt with $G = 32$ has a comparable model-FLOPs budget to eight-rollout GRPO. As shown in Table 2, in the actual four-H100 implementation, the measured wall-clock results indicate that the larger ES population does not translate into additional end-to-end overhead in this environment. This comparison should be interpreted along separate efficiency axes: Agentic ESOpt is substantially more memory efficient and competitive in model compute, while it intentionally spends more independent environment evaluations to replace reference-model evaluation and backpropagation.

Takeaway 2

Agentic ESOpt demonstrates **high model-side efficiency**, requiring only inference-level GPU memory. Although it evaluates more trajectories than Agentic GRPO ($G = 32$ versus eight rollouts), the saved reference-model and backward-pass compute keeps model FLOPs and measured wall-clock time comparable in Sudoku, making full-parameter adaptation of the 27B agent in Table 4 practical.

5 Agentic ESOpt for Train-Time Fine-tuning

Besides the hand-crafted Sudoku, this section evaluates the ability of Agentic ESOpt in fine-tuning LLMs towards specific-purpose agents. We fine-tune agents for Python computation tool usage in Agentic Math reasoning and OCR and document-image analysis tool in DocVQA, and towards a web agent in WebArena.

5.1 Agentic ReAct-Style Tool Usage on Math and DocVQA

We first fine-tune LLMs for the tool-use ability in ReAct-style interaction (Yao et al., 2022). This task is a relatively long-horizon scenario with a usual horizon $H > 10$. We follow the setting and hyperparameters in Ni et al. (2026), training Math on 400 DAPO problems and evaluating it on 100 held-out DAPO problems and out-of-distribution 30-problem AIME 2026. For DocVQA, we fine-tune agents on a 50-question validation subset and evaluate them on held-out 100 questions. In evaluating all methods, we set the horizon limit to 50 turns for both MATH and DocVQA, confining the max_token in each turn to 4096 for MATH and 512 for DocVQA, respectively. This setting will not cause significant truncation. We implement two fine-tuning methods, Agentic GRPO and Agentic ESOpt, on Qwen3.5-4B. Besides, we also implement a prompt-space optimization method, Trace2Skill, which collects trajectories by sampling and synthesizes trajectories for skills at the end. We also implement a sequential combination of fine-tuning methods and Trace2Skill, where Agentic GRPO + Trace2Skill and Agentic ESOpt + Trace2Skill represent using the trajectories collected during training over No Skill prompts for skill distillation. GRPO is 8-rollout as well, and we have $G = 16$ for Agentic ESOpt and 16 samples for Trace2Skill. As discussed in Appendix C.5, in fine-tuning agents, Agentic ESOpt consumes only approximately half the FLOPs of GRPO. Detailed settings are given in Sections D.2 and D.3.

Table 3 Math-reasoning and DocVQA results under No Skill and Trace2Skill contexts. Qwen3.5-27B No Skill is an evaluation-only baseline; all optimized rows use Qwen3.5-4B. DAPO, AIME 2026, and DocVQA accuracy are reported in %. Mean@4 averages four sampled Pass@1 accuracies; Pass@4 or Max@4 represent the best of four samples.

Model	Method	Math reasoning				DocVQA			
		DAPO		AIME 2026		ANLS		Accuracy	
		Mean@4	Pass@4	Mean@4	Pass@4	Mean@4	Max@4	Mean@4	Pass@4
Qwen3.5-27B	No Skill	65.8	87.0	76.7	93.3	0.5036	0.7843	51.8	69.0
Qwen3.5-4B	No Skill	63.0	86.0	55.8	86.7	0.3875	0.5981	40.3	53.0
Qwen3.5-4B	Agentic GRPO + No Skill	68.8	83.0	58.3	76.7	0.4627	0.5398	48.0	56.0
Qwen3.5-4B	Agentic ESOpt + No Skill	76.8	86.0	70.8	96.7	0.5043	0.6507	52.5	61.0
	Δ vs No Skill	$\uparrow 13.8$	0.0	$\uparrow 15.0$	$\uparrow 10.0$	$\uparrow 0.1168$	$\uparrow 0.0526$	$\uparrow 12.3$	$\uparrow 8.0$
Qwen3.5-4B	Trace2Skill	64.8	82.0	50.8	83.3	0.4612	0.6772	47.3	69.0
Qwen3.5-4B	Agentic GRPO + Trace2Skill	67.8	85.0	50.0	80.0	0.4743	0.5692	49.5	60.0
Qwen3.5-4B	Agentic ESOpt + Trace2Skill	77.3	86.0	71.7	96.7	0.5086	0.6654	52.8	61.0
	Δ vs Trace2Skill	$\uparrow 12.5$	$\uparrow 4.0$	$\uparrow 20.8$	$\uparrow 13.3$	$\uparrow 0.0474$	$\downarrow 0.0118$	$\uparrow 5.5$	$\downarrow 8.0$

As shown in Table 3, Agentic ESOpt consistently outperforms the matched Agentic GRPO baselines across ReAct-style Math and DocVQA. Without evolved skills, Agentic ESOpt improves the *Qwen3.5-4B* base model by 13.8 and 15.0 percentage points on DAPO and AIME 2026 Mean@4, respectively, and improves DocVQA Mean@4 accuracy by 12.3 points. Averaged across these three metrics, Agentic ESOpt improves the base model by 13.7 points and Agentic GRPO by 8.3 points. Agentic ESOpt also composes effectively with Trace2Skill: the combined method achieves the strongest Qwen3.5-4B Mean@4 results on DAPO, AIME 2026, and DocVQA, showing the flexibility of parameter-space optimization methods to compose with external skill optimization.

Beyond the average-performance gains, Agentic ESOpt also preserves strong Pass@4 performance. We show the full Pass@K performance in Appendix D.3.4 up to $k = 32$. Across datasets, Agentic ESOpt variants improve every reported Pass@K metric over their matched GRPO baselines (Yue et al., 2025). In AIME 2026 and DocVQA, Agentic ESOpt improves the Pass@K performance compared to the base LLM. This indicates that Agentic ESOpt performs a different and more favorable optimization paradigm compared to Agentic GRPO.

Takeaway 3

Agentic ESOpt improves average performance without sacrificing Pass@K coverage: across Math and DocVQA, both Agentic ESOpt variants outperform their matched Agentic GRPO baselines on every reported Pass@4 metric, suggesting broader coverage of successful trajectories after parameter optimization.

5.2 Agentic Reasoning for WebArena: Experiments on Model Scalability

We then evaluate **Agentic ESOpt** on WebArena-Lite, a 165-task browser benchmark derived from WebArena (Liu et al., 2025c). Each task provides a natural-language goal and an interactive website state. The agent observes a WebRL-style textual browser representation, acts in an id-based action space, and receives task-success feedback after completing a trajectory (Zhou et al., 2024; Qi et al., 2025). The evaluation set contains 21 Reddit, 32 GitLab, 35 CMS, 28 Map, 46 OSS, and 3 Wikipedia tasks; the five major categories are shown separately in Table 4, while Wikipedia is only included in the dataset average.

This setting scales Agentic ESOpt to Qwen3.5-27B on four NVIDIA H100 80GB GPUs. At this model scale, full-parameter Agentic RL is no longer practical on four H100 80GB GPUs. In contrast, Agentic ESOpt retains inference-level memory requirements, providing feasibility for us to perform full-parameter adaptation of a 27B web agent. This experiment therefore focuses on large-model feasibility and real-world agent adaptation, complementing the controlled ES-versus-RL comparison in Section 4 and 5.1. We compare two paired settings using the same Qwen3.5-27B backbone: No Skill versus Agentic ESOpt + No Skill, and Trace2Skill versus Agentic ESOpt + Trace2Skill. Both Agentic ESOpt variants share the same parameter updates learned without skills; the combined variant additionally distills a skill from the resulting trajectories, without a second skill-conditioned ES stage or alternating joint optimization. We also report closed-source GPT-5.4, GPT-5.4-mini, and GPT-5.4-nano as reference points. Complete setup details and evaluation curves are provided in Appendix D.4 and D.4.4, while task prompts and distilled skills are provided in E.4, E.4.2, and E.4.3.

All the Qwen3.5-27B rollouts for Agentic ESOpt, Trace2skill, and evaluation use temperature 0.7, a 2048-token generation budget, WebRL-style observations, and WebRL id-based actions. Agentic ESOpt sets the population size $G = 8$, uses task success on sampled training cases as the reward. Trace2skill also samples each question for 8 runs. Final results are evaluated on the official 165-task WebArena-Lite evaluation set (Liu et al., 2025c).

Table 4 WebArena-Lite success rates (%). We report the average over 3 runs and the standard deviations. Improved Agentic ESOpt cells relative to their paired baseline are shaded green; bold denotes the best result in each column. The five displayed site categories contain 162 tasks; the remaining Wikipedia tasks are included in Dataset Avg.

Model	Method	Reddit (21)	GitLab (32)	CMS (35)	Map (28)	OSS (46)	Dataset Avg.
<i>Strong frozen baseline (evaluation only)</i>							
GPT-5.4	No Skill	47.62	46.88	46.67	19.05	21.01	34.14 $\pm$ 0.76
GPT-5.4-mini	No Skill	39.68	29.17	30.48	13.10	13.77	23.23 $\pm$ 1.14
GPT-5.4-nano	No Skill	39.68	27.08	19.05	11.90	8.70	18.79 $\pm$ 0.99
Qwen3.5-27B	No Skill	50.79	35.42	41.90	8.33	21.01	29.47 $\pm$ 1.14
Qwen3.5-27B	Agentic ESOpt + No Skill	49.21	43.75	49.52	14.29	30.43	36.16 $\pm$ 0.70
	Δ vs No Skill	$\downarrow 1.58$	$\uparrow 8.33$	$\uparrow 7.62$	$\uparrow 5.96$	$\uparrow 9.42$	$\uparrow 6.69$
Qwen3.5-27B	Trace2Skill	49.21	39.58	46.67	13.10	28.26	33.94 $\pm$ 3.37
Qwen3.5-27B	Agentic ESOpt + Trace2Skill	52.80	41.67	50.48	10.71	32.61	36.36 $\pm$ 0.86
	Δ vs Trace2Skill	$\uparrow 3.59$	$\uparrow 2.09$	$\uparrow 3.81$	$\downarrow 2.39$	$\uparrow 4.35$	$\uparrow 2.42$

As shown in Table 4, Agentic ESOpt improves the Qwen3.5-27B No Skill baseline from 29.47% to 36.16%, a gain of 6.69 percentage points. When combined with Trace2Skill, it further improves the Trace2Skill baseline from 33.94% to 36.36%, demonstrating the flexibility of Agentic ESOpt in composing with skill-space optimization on larger LLMs.

6 Agentic ESOpt on Test-Time Compute: Automatic Heuristic Design

Traditional test-time compute generally fixes the LLM parameters throughout the search process, which may limit optimization when solving the task requires changing the underlying policy itself. Due to the high flexibility of Agentic ESOpt, it can incorporate test-time compute to improve its performance (Snell et al., 2024).

In this section, we evaluate the ability of Agentic ESOpt to boost a typical test-time compute problem—automatic heuristic design (AHD) (Liu et al., 2024; Zheng et al., 2025b; Huang et al., 2026; Liu et al., 2025a). Among them, the representative work EoH (Liu et al., 2024) improves the performance of these algorithms on a fixed training dataset by maintaining a population of heuristic algorithms and applying crossover and mutation operators. In generating each new heuristic, it provides an LLM agent with a problem description and a Python function signature; the LLM generates heuristic code, and the evaluator returns a scalar objective after executing the code within a solver. Agentic ESOpt can be inserted into existing EoH and pure sample procedures without changing their outer search scaffold. Agentic ESOpt + Sample or EoH therefore supports both heuristic-space

search and parameter-space adaptation. We consider two AHD scenarios and compare Sample and EoH with their corresponding Agentic ESOpt variants under the same evaluation budgets. In constructive AHD, the heuristic makes local decisions for NP-hard combinatorial optimization problems (defined in Appendix C.4): TSP, KP, and ASP; In ACO-style AHD, the algorithm provides heuristic information to an ant-colony optimizer for TSP, CVRP, and BPP. TSP, CVRP, and BPP are minimized, whereas KP and ASP are maximized.

For constructive tasks, the *Optimal* row reports the known optimum x^* only as a reference. We define the normalized optimality gap as $g(x) = \frac{|x-x^*|}{|x^*|}$, and report the gain of an Agentic ESOpt result m over its matched baseline b as $\Delta = \frac{g(b)}{g(m)} - 1$. The formal heuristic-space view, complete search settings, ACO-style results, ablations, and exact generation prompt are indexed in Appendix C.4, D.5, D.5.2, and E.5.

For Agentic ESOpt + EoH, all objectives are represented internally as minimization costs, and the reward is the selected parent’s cost minus the perturbed child’s cost. For Agentic ESOpt + Sample, the reward is the negative child cost. Rewards are z-score normalized within each parameter-update batch. Agentic ESOpt preserves the original EoH operators and attaches parameter updates only to the mutation operators $m1$ and $m2$. We report the best candidate from the final generation and average over repeated runs. A 20-run significance analysis on representative TSP and KP settings is provided in Table 18.

Table 5 Design constructive heuristics at total evaluation $T \in \{1000, 2000\}$. TSP is minimized; KP and ASP are maximized. The Optimal row shows optimum objective values, not an experimental baseline. Sample and EoH are each paired with their Agentic ESOpt counterpart; the Δ rows report the gap-ratio gain defined in the text. Baselines and Agentic ESOpt use **LLaMA-3.1-8B-Instruct** for all runs.

Method	TSP $N = 20$	TSP $N = 50$	KP $N = 50, W = 12.5$	KP $N = 100, W = 25$	ASP $N = 15, W = 10$	ASP $N = 21, W = 15$
Optimal	3.8199	5.6750	20.0370	40.2710	3,003	43,596
Greedy Construct	4.4797	6.9590	19.9850	40.2250	1,530	15,050
<i>Total Evaluations: $T = 1000$</i>						
Sample	4.3286	6.7110	19.9896	40.2297	2,753	30,336.67
Agentic ESOpt + Sample	4.2336	6.5488	19.9899	40.2314	2,729	31,082.67
Δ vs Sample	↓22.96%	↓18.56%	↑0.79%	↑4.42%	↓8.76%	↑5.96%
EoH	4.2481	6.5450	19.9958	40.2320	2,760	28,465.67
Agentic ESOpt + EoH	4.2170	6.4631	20.0007	40.2351	2,770	30,887.33
Δ vs EoH	↓7.83%	↓10.39%	↑13.59%	↑8.65%	↑4.67%	↑29.18%
<i>Total Evaluations: $T = 2000$</i>						
Sample	4.2585	6.6008	19.99118	40.2320	2,759	30,123.33
Agentic ESOpt + Sample	4.2098	6.5332	19.99120	40.2320	2,751	30,269.33
Δ vs Sample	↓12.51%	↓7.88%	↑0.04%	0.00%	↓3.17%	↑1.10%
EoH	4.2165	6.4706	19.9984	40.2356	2,764	28,016.00
Agentic ESOpt + EoH	4.1799	6.4442	19.9987	40.2358	2,784	30,124.67
Δ vs EoH	↓10.19%	↓3.43%	↑0.81%	↑0.48%	↑9.92%	↑24.36%

As shown in Table 5, Agentic ESOpt + EoH improves all six constructive test sets at both evaluation budgets. Agentic ESOpt + Sample improves nine of twelve comparisons, with one tie and two regressions. Across both constructive baselines, Agentic ESOpt therefore improves 21 of 24 matched comparisons. The ACO-style results and additional ablations are reported in Section D.5.2. Combining the constructive and ACO-style settings, Agentic ESOpt improves 28 of 36 matched method–budget comparisons across 12 test sets and six scenarios.

Takeaway 4

AHD demonstrates that Agentic ESOpt can extend test-time compute from search over a frozen LLM policy to **coupled heuristic–parameter optimization**. Without modifying the outer Sample or EoH search scaffold and under matched evaluation budgets, Agentic ESOpt improves **28 of 36** comparisons, showing that lightweight black-box parameter adaptation can systematically enhance existing test-time search.

7 Discussion

We use this section to clarify the regime in which parameter-space ES is most attractive, the current scope of the evidence, and the main directions for extending Agentic ESOpt.

7.1 Population Scaling with Model Size

The population size G is a key hyperparameter of ES. Our current results provide preliminary evidence that stronger backbones may be less sensitive to small populations. As shown in Table 6, for the 4B model, increasing the population from $G = 8$ to $G = 16$ on the same 15-turn Sudoku setting increases the best test accuracy from 5.10% to 35.42%, and final test success increases from 2.95% to 22.92%. However, the 9B model is substantially less dependent on a large population. Increasing G from 8 to 16 changes best test success from 30.21% to 37.50%, a 24.1% relative improvement, while final test success remains 30.21%, giving no relative improvement.

Table 6 Vanilla-ES population sensitivity on 15-turn Sudoku. Relative changes compare $G = 16$ with $G = 8$.

Backbone	G	Best test	Final test	Δ best test	Δ final test
Qwen3.5-4B	8	5.10	2.95	—	—
Qwen3.5-4B	16	35.42	22.92	+594.5%	+677.0%
Qwen3.5-9B	8	30.21	30.21	—	—
Qwen3.5-9B	16	37.50	30.21	+24.1%	0.0%

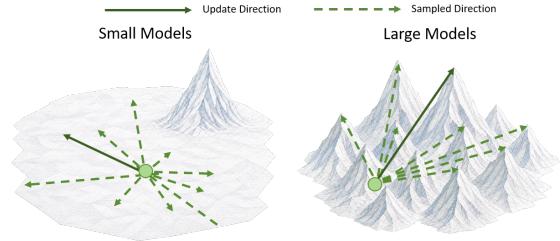


Figure 4 Intuition for population scaling: sampled directions around a stronger backbone are more likely to align with a useful Agentic ESOpt direction.

As shown in Figure 4, one plausible interpretation is that a pre-trained stronger backbone will lead to a more competent local region, so a larger fraction of nearby perturbations produce behavior informative for the update. This interpretation is consistent with the finding of Gan and Isola (2026) that useful behavioral diversity becomes denser around stronger pre-trained models. This supports the prospect of using Agentic ESOpt to fine-tune larger advanced LLMs with fewer FLOPs; we will take establishing a universal scaling law and implementing Agentic ESOpt on frontier LLMs as future work.

Takeaway 5

The experiment provides **initial population-sensitivity evidence**: doubling G changes final-test success by +677.0% for 4B but 0.0% for 9B. That indicates stronger backbones may need fewer sampled directions.

8 Conclusion

This paper presents **Agentic ESOpt**, a full-parameter evolution-strategy framework for fine-tuning long-horizon LLM agents requiring only minimal inference-level GPU memory. We argue that **for long-horizon agents, Agentic ESOpt is a more natural optimization paradigm than conventional Agentic RL**. As interaction horizons grow and feedback becomes sparse, policy gradients must assign a trajectory-level outcome across an expanding sequence of actions. Agentic ESOpt instead attributes the outcome to a coherent policy perturbation, avoiding explicit horizon-wise action-score accumulation.

Our experiments support this shift. On controlled Sudoku, Agentic PPO and Agentic GRPO remain competitive at shorter horizons, whereas **Agentic ESOpt becomes strongest as the minimum successful horizon grows**; the advantage further extends to ReAct-style Math and DocVQA. Agentic ESOpt also provides a model-scaling advantage through forward-only optimization. Its **inference-level memory** footprint enables full-parameter adaptation of a Qwen3.5-27B WebArena agent on four H100 GPUs.

Beyond train-time fine-tuning, its **flexibility** naturally integrates parameter adaptation into broader optimization loops of test-time compute. Agentic ESOpt improves skill optimization and test-time heuristic search, including 28 of 36 matched AHD comparisons under fixed evaluation budgets.

Taken together, these results position ES not as a cheaper substitute for RL, but as **a better-matched optimization mechanism for long-horizon, sparse-feedback LLM agents**. As agentic systems move toward longer interactions, larger models, and increasingly complex prompt-space interactive components, Agentic ESOpt may become a central paradigm for agent fine-tuning and online self-improvement.

Limitations of the current study are discussed in Appendix A.1, and future work is outlined in Appendix A.2.

Acknowledgement

We would like to sincerely thank Jiaying Wu, Penghui Qi, Zichen Liu, and Ziqiao Meng from the National University of Singapore, as well as Ziang Li from humans& for their important comments on the methodology and paper-writing.

References

- Tianzhe Chu, Yuexiang Zhai, Jihan Yang, Shengbang Tong, Saining Xie, Dale Schuurmans, Quoc V Le, Sergey Levine, and Yi Ma. Sft memorizes, rl generalizes: A comparative study of foundation model post-training. *arXiv preprint arXiv:2501.17161*, 2025.
- Shangheng Du, Jiabao Zhao, Jinxin Shi, Zhentao Xie, Xin Jiang, Yanhong Bai, and Liang He. A survey on the optimization of large language model-based agents. *ACM Computing Surveys*, 58(9):1–37, 2026.
- Lang Feng, Zhenghai Xue, Tingcong Liu, and Bo An. Group-in-group policy optimization for llm agent training. *Advances in Neural Information Processing Systems*, 38:46375–46408, 2026.
- Zihang Fu, Haonan Wang, Jian Kang, Kenji Kawaguchi, and Jiaying Wu. Reasoning resides in layers: Restoring temporal reasoning in video-language models with layer-selective merging. *arXiv preprint arXiv:2604.11399*, 2026.
- Yulu Gan and Phillip Isola. Neural thickets: Diverse task experts are dense around pretrained weights. *arXiv preprint arXiv:2603.12228*, 2026.
- Gemini Team. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*, 2025.
- Yu Gu, Zhi Zheng, Yunpeng Ba, Xialiang Tong, Mingxuan Yuan, and Zhenkun Wang. Hyper-es: Effective evolution strategies for llm reasoning via descent direction merging. *arXiv preprint arXiv:2608.05541*, 2026.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Shuo He, Lang Feng, Xin Cheng, Lei Feng, Bo An, et al. Hierarchy-of-groups policy optimization for long-horizon agentic tasks. In *International Conference on Learning Representations*, volume 2026, pages 27572–27593, 2026.
- Zhenyu Hou, Yujiang Li, Jie Tang, and Yuxiao Dong. Single-rollout asynchronous optimization for agentic reinforcement learning. *arXiv preprint arXiv:2607.07508*, 2026.
- William Hoy, Binxu Wang, and Xu Pan. Matching accuracy, different geometry: Evolution strategies vs grpo in llm post-training. *arXiv preprint arXiv:2604.01499*, 2026.
- Ziyao Huang, Weiwei Wu, Kui Wu, Wei-Bin Lee, and Jianping Wang. Calm: Co-evolution of algorithms and language model for automatic heuristic design. In *International Conference on Learning Representations*, volume 2026, pages 72468–72510, 2026.
- Sunghwan Kim, Junhee Cho, Beong-woo Kwak, Taeyoon Kwon, Liang Wang, Nan Yang, Xingxing Zhang, Furu Wei, and Jinyoung Yeo. On training large language models for long-horizon tasks: An empirical study of horizon length. *arXiv preprint arXiv:2605.02572*, 2026.
- Daria Korotyshova, Boris Shaposhnikov, Alexey Malakhov, Alexey Khokhulin, Nikita Surnachev, Kirill Ovcharenko, George Bredis, Alexey Gorbатовski, Viacheslav Sinii, and Daniil Gavrilov. Essa: Evolutionary strategies for scalable alignment. *arXiv preprint arXiv:2507.04453*, 2025.
- Junbo Li, Peng Zhou, Rui Meng, Meet P Vadera, Lihong Li, and Yang Li. Turn-ppo: Turn-level advantage estimation with ppo for improved multi-turn rl in agentic llms. In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 6227–6243, 2026.
- Fei Liu, Xialiang Tong, Mingxuan Yuan, Xi Lin, Fu Luo, Zhenkun Wang, Zhichao Lu, and Qingfu Zhang. Evolution of heuristics: Towards efficient automatic algorithm design using large language model. *arXiv preprint arXiv:2401.02051*, 2024.
- Fei Liu, Rui Zhang, Xi Lin, Zhichao Lu, and Qingfu Zhang. Fine-tuning large language model for automated algorithm design. *arXiv preprint arXiv:2507.10614*, 2025a.
- WenTao Liu, Siyu Song, Hao Hao, and Aimin Zhou. Ea4llm: A gradient-free approach to large language model optimization via evolutionary algorithms. *arXiv preprint arXiv:2510.10603*, 2025b.

- Xiao Liu, Tianjie Zhang, Yu Gu, Iat Long Iong, Song XiXuan, Yifan Xu, Shudan Zhang, Hanyu Lai, Jiadai Sun, Xinyue Yang, et al. Visualagentbench: Towards large multimodal models as visual foundation agents. In *International Conference on Learning Representations*, volume 2025, pages 95650–95707, 2025c.
- Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding r1-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025d.
- Zichen Liu, Anya Sims, Keyu Duan, Changyu Chen, Simon Yu, Xiangxin Zhou, Haotian Xu, Shaopan Xiong, Bo Liu, Chenmien Tan, et al. Gem: A gym for agentic llms. *arXiv preprint arXiv:2510.01051*, 2025e.
- Zeyao Ma, Bohan Zhang, Jing Zhang, Jifan Yu, Xiaokang Zhang, Xiaohan Zhang, Sijia Luo, Xi Wang, and Jie Tang. Spreadsheetbench: Towards challenging real world spreadsheet manipulation. *Advances in Neural Information Processing Systems*, 37:94871–94908, 2024.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems*, 36:46534–46594, 2023.
- Sadhika Malladi, Tianyu Gao, Eshaan Nichani, Alex Damian, Jason D Lee, Danqi Chen, and Sanjeev Arora. Fine-tuning language models with just forward passes. *Advances in Neural Information Processing Systems*, 36:53038–53075, 2023.
- Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Erchao Zhao, Xiaoxi Jiang, and Guanjun Jiang. Trace2skill: Distill trajectory-local lessons into transferable agent skills. *arXiv preprint arXiv:2603.25158*, 2026.
- Zehan Qi, Xiao Liu, Iat Long Iong, Hanyu Lai, Xueqiao Sun, Jiadai Sun, Xinyue Yang, Yu Yang, Shuntian Yao, Wei Xu, et al. Webrl: Training llm web agents via self-evolving online curriculum reinforcement learning. In *International Conference on Learning Representations*, volume 2025, pages 79791–79821, 2025.
- Xin Qiu, Yulu Gan, Conor F. Hayes, Qiyao Liang, Yinggan Xu, Roberto Dailey, Elliot Meyerson, Babak Hodjat, and Risto Miikkulainen. Evolution strategies at scale: LLM fine-tuning beyond reinforcement learning. In *International Conference on Machine Learning*, 2026.
- Tim Salimans, Jonathan Ho, Xi Chen, Szymon Sidor, and Ilya Sutskever. Evolution strategies as a scalable alternative to reinforcement learning. *arXiv preprint arXiv:1703.03864*, 2017.
- Bidipta Sarkar, Mattie Fellows, Juan Agustin Duque, Alistair Letcher, Antonio León Villares, Anya Sims, Clarisse Wibault, Dmitry Samsonov, Dylan Cope, Jarek Liesen, et al. Evolution strategies at the hyperscale. *arXiv preprint arXiv:2511.16652*, 2025.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Yifei Shen, Bo Li, and Xinjie Zhang. Skillopt-lite: Better and faster agent self-evolution via one line of vibe. *arXiv preprint arXiv:2607.03451*, 2026.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.
- Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.
- Mingyang Song and Mao Zheng. A survey of on-policy distillation for large language models. *arXiv preprint arXiv:2604.00626*, 2026.
- Zhishen Sun, Sizhe Dang, Guang Dai, and Haishan Ye. Essam: A novel competitive evolution strategies approach to reinforcement learning for memory efficient llms fine-tuning. *arXiv preprint arXiv:2602.01003*, 2026.
- Fahim Tajwar, Guanning Zeng, Yueer Zhou, Yuda Song, Daman Arora, Yiding Jiang, Jeff Schneider, Ruslan Salakhutdinov, Haiwen Feng, and Andrea Zanette. Maximum likelihood reinforcement learning. *arXiv preprint arXiv:2602.02710*, 2026.

- Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, Jianfeng Cai, Xinyuan Cai, Peizhou Cao, Yuxuan Cao, Ziwei Chai, Y Charles, et al. Kimi k3: Open frontier intelligence. *arXiv preprint arXiv:2607.24653*, 2026.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, et al. OpenHands: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.
- Zixuan Wang, Yuchen Yan, Hongxing Li, Teng Pan, Dingming Li, Ruiqing Zhang, Weiming Lu, Jun Xiao, Yueting Zhuang, and Yongliang Shen. Milestone-guided policy learning for long-horizon language agents. *arXiv preprint arXiv:2605.06078*, 2026.
- Yaxiong Wu, Sheng Liang, Chen Zhang, Yichao Wang, Yongyue Zhang, Huifeng Guo, Ruiming Tang, and Yong Liu. From human memory to ai memory: A survey on memory mechanisms in the era of llms. *arXiv preprint arXiv:2504.15965*, 2025.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. *arXiv preprint arXiv:2502.12110*, 2025.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.
- Yifan Yang, Ziyang Gong, Weiquan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, et al. Skillopt: Executive strategy for self-evolving agent skills. *arXiv preprint arXiv:2605.23904*, 2026.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822, 2023.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in llms beyond the base model? *arXiv preprint arXiv:2504.13837*, 2025.
- Chenchen Zhang. From reasoning to agentic: Credit assignment in reinforcement learning for large language models. *arXiv preprint arXiv:2604.09459*, 2026.
- YanJun Zhao, Sizhe Dang, Haishan Ye, Guang Dai, Yi Qian, and Ivor Tsang. Second-order fine-tuning without pain for llms: A hessian informed zeroth-order optimizer. In *International Conference on Learning Representations*, volume 2025, pages 43496–43520, 2025.
- Zhi Zheng, Yu Gu, Wei Liu, Yee Whye Teh, and Wee Sun Lee. Soft-grpo: Surpassing discrete-token llm reinforcement learning via gumbel-reparameterized soft-thinking policy optimization. *arXiv preprint arXiv:2511.06411*, 2025a.
- Zhi Zheng, Zhuoliang Xie, Zhenkun Wang, and Bryan Hooi. Monte carlo tree search for comprehensive exploration in llm-based automatic heuristic design. *arXiv preprint arXiv:2501.08603*, 2025b.
- Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. Webarena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, volume 2024, pages 15585–15606, 2024.
- King Zhu, Hanhao Li, Siwei Wu, Tianshun Xing, Dehua Ma, Xiangru Tang, Minghao Liu, Jian Yang, Jiaheng Liu, Yuchen Eleanor Jiang, et al. Scaling test-time compute for llm agents. *arXiv preprint arXiv:2506.12928*, 2025.

Appendix

Appendix Contents

A. Limitations & Future Work	16
A.1. Limitations	16
A.2. Future Work	16
B. Additional Related Work	17
B.1. Agentic Reinforcement Learning	17
B.2. Evolution Strategies	17
B.3. Skill and Memory-Based Self-Improvement	17
C. Theoretical Analysis and Computational Accounting	18
C.1. Derivation of the Scalar-Score ES Gradient	18
C.2. Proof of the Gaussian-Smoothing Bias Expansion	18
C.3. Long-Horizon Credit Assignment	19
C.4. AHD as Heuristic-Space Optimization	20
C.5. Training FLOPs Calculation	21
D. Experimental Settings and Additional Results	22
D.1. Sudoku	22
D.2. Math Reasoning	24
D.3. DocVQA	25
D.4. WebArena-Lite	27
D.5. Automatic Heuristic Design	28
D.6. Cross-Setting Hyperparameter Summary	30
E. Prompts and Skills	32
E.1. Sudoku	32
E.2. Math Reasoning	32
E.3. DocVQA	33
E.4. WebArena-Lite	34
E.5. Automatic Heuristic Design	36
F. Licensing and External Components	37
F.1. Licensing Scope	37
F.2. Upstream Repositories Used by the Released Workflow	37

A Limitations & Future Work

A.1 Limitations

Introducing new hyperparameters. Compared to Agentic RL, Agentic ESOpt introduces hyperparameters for the perturbation radius σ . Their optimal values can depend on LLMs, reward distribution, and environment, which may introduce additional tuning cost compared with using a fixed training recipe. However, as summarized in Appendix D.6, we use relatively consistent configurations across tasks and LLMs, taking $\sigma_0 \approx 1e - 3$ and $\alpha \approx 5e - 4$ across 5 experiments. This shows that this set of parameters is highly generic. We will take the automatic schedule for σ as future work.

Concerns in Expensive Evaluation Scenarios. Agentic ESOpt trades backpropagation cost for a larger number of independent environment evaluations: under matched model FLOPs, it can afford more trajectories because each trajectory requires only a forward pass. This trade-off may become less favorable when environment evaluation itself is *extremely* expensive, in which case rollout cost rather than model computation can dominate the total budget.

Continual learning remains unclear. The current experiments establish in-setting adaptation. As mentioned in Hoy et al. (2026), unlike GRPO, which usually does sparse updates, ES will lead to a random walk in directions irrelevant to the optimization objective, raising concerns about continual learning. However, although the ES update is dense in the strict sense, its magnitude is still highly concentrated. As shown in Table 7, Agentic ESOpt updates can still maintain sparsity within a certain threshold: 96.26% of the final parameter updates remain within the perturbation scale σ , and 99.42% are smaller than 2.0×10^{-3} .

Table 7 Distribution of parameter-update magnitudes after Agentic ESOpt for WebArena on Qwen3.5-27B with perturbation scale $\sigma_t = 1.5 \times 10^{-3}$.

Update-magnitude threshold	Fraction of parameters
$ \Delta\theta \leq 1.0 \times 10^{-3}$	83.68%
$ \Delta\theta \leq 1.5 \times 10^{-3} (= \sigma)$	96.26%
$ \Delta\theta \leq 2.0 \times 10^{-3}$	99.42%

A.2 Future Work

Scaling to advanced LLMs and population scaling laws. The inference-level memory requirement of Agentic ESOpt provides a direct path toward full-parameter adaptation of substantially larger LLM agents. Our 4B/9B results further suggest that stronger backbones may require fewer perturbation directions, motivating a systematic study of population scaling with model capability. Moreover, as future work, scaling to advanced LLMs (Team et al., 2026; Guo et al., 2025; Gemini Team, 2025) may also enable online adaptation in industrial scenarios involving proprietary tools, APIs, and complex workflows.

Quantized ES optimization. Quantization could further extend the scalability of Agentic ESOpt, but applying dense parameter perturbations directly to quantized weights requires scale-aware noise generation and numerically stable updates (Sarkar et al., 2025). Developing ES-specific infra for quantization-compatible perturbation and seed-replay mechanisms is therefore an important systems direction for large-scale ES fine-tuning.

Tighter skill-parameter co-evolution. Our experiments demonstrate both shared-rollout skill distillation with Trace2Skill and online parameter adaptation inside EoH. A natural next step is fully coupled multi-step skill optimization (Yang et al., 2026; Shen et al., 2026) in which the external context c_t and model parameters θ_t evolve on comparable timescales, allowing skill- and parameter-space updates to continually reshape the data distribution seen by one another.

B Additional Related Work

B.1 Agentic Reinforcement Learning

Reinforcement learning provides the standard route for optimizing model parameters from verifiable task feedback (Shao et al., 2024; Liu et al., 2025d; Yu et al., 2025; Zheng et al., 2025a; Tajwar et al., 2026). These methods have demonstrated strong performance on single-turn LLM reasoning.

The recent agentic scenarios bring new challenges about long-context and long-horizon credit assignment (Kim et al., 2026; Liu et al., 2025e), and there are studies that try to apply RL techniques to agentic scenarios (Qi et al., 2025; Feng et al., 2026; Wang et al., 2026). Some work highlights better rollout in a GRPO-style baseline (Feng et al., 2026; He et al., 2026), and there are also works training critic network and use PPO for better credit assignment (Hou et al., 2026). These methods can improve the credit assignment ability of RL, but we believe that considering Agentic ESOpt should be seen as a more direct solution. Gradient-based agentic RL usually requires a training stack that stores rollouts, estimates token-level policy gradients, manages reference models or KL constraints, and consumes substantial memory. Agentic ESOpt maintains the same high-level goal of parameter adaptation, but uses scalar black-box fitness so that the environment trajectory does not need to be differentiable or retained for backpropagation.

B.2 Evolution Strategies for single-turn LLM Fine-Tuning.

As advanced LLMs are equipped with larger and larger parameters, fine-tuning them on devices with moderate scales becomes unaffordable, and ES have emerged as efficient gradient-free optimizers for fine-tuning single-turn LLMs. Qiu et al. (2026) demonstrated that ES can scale to full-parameter LLM fine-tuning, matching or exceeding GRPO in sample efficiency and training stability. This momentum has driven ES-based methods into pre-training (Liu et al., 2025b), few-shot adaptation (Gan and Isola, 2026; Korotyshova et al., 2025; Gu et al., 2026; Fu et al., 2026), and memory-efficient tuning via sharpness-aware mechanisms (Sun et al., 2026). In these methods, ES achieves higher GPU memory efficiency but lower performance than RL methods. However, we argue that the structural advantages of ES are actually pronounced in fine-tuning long-horizon agents, where Agentic ESOpt can become preferable to Agentic RL, instead of merely serving as a cheaper alternative.

As a similar category of gradient-free LLM fine-tuning, zeroth-order methods (Malladi et al., 2023; Zhao et al., 2025) focus on SFT, making them hard to apply to single-turn and multi-turn reasoning scenarios.

B.3 Skill and Memory-Based Self-Improvement and Test-Time Compute

A broad class of self-improving agents improves behavior by optimizing external, non-parametric components while keeping the underlying LLM fixed. Reflexion converts failed trajectories into verbal feedback that conditions later trials (Shinn et al., 2023); Voyager accumulates executable skills for open-ended embodied exploration (Wang et al., 2023); and recent memory systems organize and retrieve past experience over long-horizon interactions (Xu et al., 2025; Wu et al., 2025). More recent methods such as SkillOpt and Trace2Skill make this optimization explicit by treating the skill document itself as an optimizable object (Yang et al., 2026; Ni et al., 2026).

Test-time compute (Snell et al., 2024; Zhu et al., 2025) methods follow a related principle: they improve task performance through additional search (Yao et al., 2023), sampling, reflection (Madaan et al., 2023), or evolution (Liu et al., 2024) at inference time, while typically leaving the model parameters unchanged. These approaches are lightweight and flexible, but their optimization remains constrained by the behaviors accessible to the frozen policy.

Agentic ESOpt complements these methods by introducing parameter adaptation as an additional optimization dimension. Its black-box update can reuse the same trajectory-level feedback already collected for skill, memory, or test-time search, allowing external-space optimization and parameter-space optimization to proceed together. This enables Agentic ESOpt to strengthen existing self-improvement and test-time compute pipelines without replacing their original search or skill-update mechanisms.

C Theoretical Analysis and Computational Accounting

C.1 Derivation of the Scalar-Score ES Gradient

Let d be the number of LLM parameters and let

$$q_\sigma(\vartheta \mid \theta) = \frac{1}{(2\pi\sigma^2)^{d/2}} \exp\left(-\frac{\|\vartheta - \theta\|_2^2}{2\sigma^2}\right)$$

denotes the density of a Gaussian perturbation centered at θ . Equivalently, $\vartheta = \theta + \sigma\epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$. The smoothed objective can then be written as

$$J_\sigma(\theta; c) = \int J(\vartheta; c) q_\sigma(\vartheta \mid \theta) d\vartheta.$$

Assuming differentiation and integration can be interchanged, the log-derivative identity gives

$$\nabla_\theta J_\sigma(\theta; c) = \int J(\vartheta; c) \nabla_\theta q_\sigma(\vartheta \mid \theta) d\vartheta \quad (5)$$

$$= \mathbb{E}_{\vartheta \sim q_\sigma(\cdot \mid \theta)} [J(\vartheta; c) \nabla_\theta \log q_\sigma(\vartheta \mid \theta)]. \quad (6)$$

For the Gaussian density,

$$\nabla_\theta \log q_\sigma(\vartheta \mid \theta) = \frac{\vartheta - \theta}{\sigma^2} = \frac{\epsilon}{\sigma},$$

and hence

$$\nabla_\theta J_\sigma(\theta; c) = \frac{1}{\sigma} \mathbb{E}_\epsilon [J(\theta + \sigma\epsilon; c) \epsilon].$$

Now expand the inner objective using its trajectory definition:

$$J(\theta + \sigma\epsilon; c) = \mathbb{E}_{\tau \sim \pi_{\theta + \sigma\epsilon}(\cdot \mid c)} [R(\tau)].$$

The law of total expectation therefore yields

$$\nabla_\theta J_\sigma(\theta; c) = \frac{1}{\sigma} \mathbb{E}_{\substack{\epsilon \sim \mathcal{N}(0, I) \\ \tau \sim \pi_{\theta + \sigma\epsilon}(\cdot \mid c)}} [R(\tau) \epsilon].$$

Thus, for independent perturbations ϵ_i and conditional rollouts $\tau_i \sim \pi_{\theta + \sigma\epsilon_i}(\cdot \mid c)$, the canonical Monte Carlo estimator is

$$\hat{g}_{\text{ES}} = \frac{1}{G\sigma} \sum_{i=1}^G R(\tau_i) \epsilon_i, \quad \mathbb{E}[\hat{g}_{\text{ES}}] = \nabla_\theta J_\sigma(\theta; c).$$

Because $\mathbb{E}[\epsilon] = 0$, any perturbation-independent scalar baseline b may replace $R(\tau_i)$ by $R(\tau_i) - b$ without changing this expectation. Crucially, the estimator uses the environment only to produce the scalar $R(\tau_i)$; no derivative of the sampled actions, environment transitions, or reward function is required. The within-population reward standardization used by Agentic ESOpt is the practical finite-sample variant described in the main text.

C.2 Proof of the Gaussian-Smoothing Bias Expansion

This subsection proves [Lemma 1](#). We suppress the fixed external context c for readability and write $J(\theta)$ for the corresponding task objective.

Proof of Lemma 1. Define the Gaussian-smoothed objective

$$J_\sigma(\theta) = \mathbb{E}_{\epsilon \sim \mathcal{N}(0, I)} [J(\theta + \sigma\epsilon)].$$

By the Gaussian score-function identity, or equivalently integration by parts,

$$\nabla_\theta J_\sigma(\theta) = \frac{1}{\sigma} \mathbb{E}_\epsilon [J(\theta + \sigma\epsilon) \epsilon].$$

Thus, the population ES estimator is unbiased for $\nabla_\theta J_\sigma(\theta)$.

To compare the smoothed and original objectives, expand $J(\theta + \sigma\epsilon)$ around θ . Odd moments of a centered Gaussian vanish, and $\mathbb{E}[\epsilon\epsilon^\top] = I$, giving

$$J_\sigma(\theta) = J(\theta) + \frac{\sigma^2}{2} \text{Tr}(\nabla_\theta^2 J(\theta)) + O(\sigma^4) = J(\theta) + \frac{\sigma^2}{2} \Delta_\theta J(\theta) + O(\sigma^4).$$

Differentiating the expansion yields

$$\nabla_\theta J_\sigma(\theta) = \nabla_\theta J(\theta) + \frac{\sigma^2}{2} \nabla_\theta \Delta_\theta J(\theta) + O(\sigma^4).$$

Therefore, the leading smoothing bias relative to $\nabla_\theta J(\theta)$ is proportional to σ^2 , as stated in [Lemma 1](#). $\square$

C.3 Long-Horizon Credit Assignment

Let $u_t = \nabla_\theta \log \pi_\theta(a_t | s_t)$ be the policy score at turn t , and let $\tilde{R} = R - b$ denote the baseline-centered terminal return. The trajectory-level policy-gradient estimator is

$$\hat{g}_{\text{PG}} = \tilde{R} \sum_{t=1}^H u_t.$$

Assume, as a scaling approximation, that (i) the return has weak correlation with any individual action score, (ii) score terms at different turns are approximately uncorrelated, and (iii) their marginal covariance is comparable across turns, $\text{Cov}(u_t) \approx \Sigma_u$. Then

$$\text{Cov}(\hat{g}_{\text{PG}}) \approx \text{Var}(\tilde{R}) \text{Cov}\left(\sum_{t=1}^H u_t\right) \tag{7}$$

$$= \text{Var}(\tilde{R}) \left[\sum_{t=1}^H \text{Cov}(u_t) + \sum_{s \neq t} \text{Cov}(u_s, u_t) \right] \tag{8}$$

$$\approx H \text{Var}(\tilde{R}) \Sigma_u. \tag{9}$$

Thus, under these assumptions, the policy-score contribution to the estimator variance grows approximately linearly with the realized horizon H .

For Agentic ESOpt, one perturbation $\epsilon \sim \mathcal{N}(0, I)$ is sampled for the complete trajectory, giving

$$\hat{g}_{\text{ES}} = \tilde{R}(\theta + \sigma\epsilon) \frac{\epsilon}{\sigma}.$$

Under the analogous weak-correlation approximation,

$$\text{Cov}(\hat{g}_{\text{ES}}) \approx \text{Var}(\tilde{R}) \text{Cov}\left(\frac{\epsilon}{\sigma}\right),$$

whose parameter-score factor contains no sum over turns. The comparison therefore isolates an explicit horizon-dependent source of variance in action-space policy gradients that is absent from the ES parameter score.

Scope of the scaling comparison. The above argument concerns the **horizon-dependent score structure** of the two estimators. Agentic ESOpt can still become harder to optimize as the horizon grows because the return distribution may become sparser or less discriminative. Its estimator quality may also depend on the parameter dimension d , perturbation radius σ , population size G , and the local geometry of the smoothed objective. Accordingly, the analysis does not assert that the total variance of ES is independent of H , nor that ES universally dominates policy gradients. The comparative prediction is that, when other sources of difficulty are approximately comparable, increasing the effective horizon introduces an additional accumulation of action-score terms for policy gradients but not for the ES parameter score. This follows the scaling perspective of [Salimans et al. \(2017, Sec. 3.1\)](#).

From realized horizon H to minimum successful horizon H^* . The theoretical quantity above is the realized trajectory length H . In the Sudoku experiments, we instead control

$$H^*(x) = \min_{\tau: R(\tau)=1} |\tau|,$$

which gives a task-dependent lower bound on the horizon of every successful trajectory. Because each valid Sudoku action fills at most one masked cell, masking 5, 10, and 15 cells yields $H^* = 5, 10$, and 15, respectively. Failed, invalid, or unproductive actions can make the realized horizon substantially larger than H^* .

Increasing H^* in this natural task family also changes aspects of task difficulty, such as the amount of missing information and the probability of completing all required actions correctly. The Sudoku study is therefore intended to test the practical prediction that the relative behavior of parameter-space and action-space attribution changes as the minimum interaction requirement grows, instead of identifying a pure delay effect in an otherwise identical MDP. The executed-turn analysis in Figure 6 complements H^* by directly measuring how the realized horizon evolves during optimization.

C.4 AHD as Heuristic-Space Optimization

Automatic heuristic design is not a language-generation task in the usual sense: the generated text matters only through the algorithmic behavior it induces. We therefore view AHD as optimization over a heuristic space (Zheng et al., 2025b). Let $\mathcal{X}$ be a distribution over problem instances and let $h \in \mathcal{H}$ denote a heuristic sampled from the LLM-induced proposal distribution. A solver A_h uses h to construct a solution $y = A_h(x)$ for instance x . The idealized objective is

$$\min_{h \in \mathcal{H}} \mathbb{E}_{x \sim \mathcal{X}} [S(y, x)], \quad y = A_h(x),$$

where S is a signed score chosen so that larger is better. This paper covers 5 NP-hard combinatorial optimization problems, including the Traveling Salesman Problem (TSP), which seeks the shortest tour visiting all nodes exactly once; the Capacitated Vehicle Routing Problem (CVRP), which minimizes the total routing cost for capacity-constrained vehicles serving customer demands; the 0-1 Knapsack Problem (KP), which maximizes the total value of selected items under a capacity constraint; the Bin-Packing Problem (BPP), which minimizes the number of fixed-capacity bins required to pack all items; and the Admissible Set Problem (ASP), which seeks the largest subset satisfying a predefined set of feasibility constraints.

For minimization problems such as TSP, CVRP, and BPP, S is the negative objective value; for maximization problems such as KP and ASP, S is the original objective value. The relevant object is therefore not the textual form of h , but the induced mapping from problem states to algorithmic decisions.

In constructive AHD, A_h is a sequential decision rule. At a partial solution state s_t , the heuristic induces a preference over feasible actions,

$$a_t \in \arg \max_{a \in \mathcal{A}(s_t)} h(s_t, a), \quad s_{t+1} = T(s_t, a_t),$$

until a complete solution is obtained. TSP evaluates the length of the constructed tour, KP evaluates the value of the selected item set under a capacity constraint, and ASP evaluates the cardinality of the constructed admissible set. These tasks test whether Agentic ESOpt can alter the LLM proposal distribution toward heuristics whose local decisions accumulate into better global solutions.

In ACO-style AHD, A_h is stochastic. The heuristic defines an energy or desirability function over solution components, which biases a sampling distribution of the form

$$p_h(a_t \mid s_t) \propto \exp(\beta h(s_t, a_t)) \cdot \Phi_t(s_t, a_t),$$

where Φ_t denotes the non-learned search state, such as pheromone or feasibility terms, and β controls the strength of the heuristic bias. The objective is the expected quality of solutions sampled by this stochastic solver. Agentic ESOpt + EoH preserves this outer optimization problem and changes only the model-induced distribution over h .

C.5 Training FLOPs Calculation

We follow the FLOPs accounting of Gan and Isola (2026, Appendix E.1). For a model with P parameters and a trajectory containing $\bar{L}$ model-processed tokens, a forward pass requires approximately $2P\bar{L}$ FLOPs, while a backward pass requires approximately $4P\bar{L}$ FLOPs.

Let T , B , and G denote the number of training iterations, prompt batch size, and number of rollouts or perturbation directions per prompt, respectively. The method-specific training costs are

$$\begin{aligned} \text{FLOPs}_{\text{ES}} &= T_{\text{ES}} B_{\text{ES}} G_{\text{ES}} \underbrace{(2)}_{\text{policy forward}} P\bar{L}_{\text{ES}} \\ &= 2T_{\text{ES}} B_{\text{ES}} G_{\text{ES}} P\bar{L}_{\text{ES}}, \end{aligned} \tag{10}$$

$$\begin{aligned} \text{FLOPs}_{\text{GRPO}} &= T_{\text{GRPO}} B_{\text{GRPO}} G_{\text{GRPO}} \underbrace{(2 + 2 + 4)}_{\substack{\text{policy forward} \\ + \text{reference forward} \\ + \text{policy backward}}} P\bar{L}_{\text{GRPO}} \\ &= 8T_{\text{GRPO}} B_{\text{GRPO}} G_{\text{GRPO}} P\bar{L}_{\text{GRPO}}, \end{aligned} \tag{11}$$

$$\begin{aligned} \text{FLOPs}_{\text{PPO}} &= T_{\text{PPO}} B_{\text{PPO}} G_{\text{PPO}} \underbrace{(2 + 2 + 2 + 4 + 4)}_{\substack{\text{policy forward} + \text{reference forward} \\ + \text{critic forward} + \text{policy backward} + \text{critic backward}}} P\bar{L}_{\text{PPO}} \\ &= 14T_{\text{PPO}} B_{\text{PPO}} G_{\text{PPO}} P\bar{L}_{\text{PPO}}. \end{aligned} \tag{12}$$

Thus, for trajectories of equal length, Agentic ESOpt costs one policy forward pass per sampled trajectory, **compared with approximately four forward-pass equivalents for Agentic GRPO and seven for Agentic PPO**. Environment execution is excluded throughout, since the comparison measures model-side training FLOPs.

Sudoku. In the reported Sudoku comparison, Agentic ESOpt and GRPO both use 100 update rounds with a prompt batch size of 32, while Agentic PPO does 500 update rounds. Agentic ESOpt evaluates $G_{\text{ES}} = 32$ perturbation directions per prompt, whereas Agentic GRPO uses $G_{\text{GRPO}} = 8$ rollouts. Therefore, assuming equal trajectory lengths,

$$\frac{\text{FLOPs}_{\text{ES}}}{\text{FLOPs}_{\text{GRPO}}} \approx \frac{2 \times 32}{8 \times 8} = 1.$$

Hence, the fourfold larger ES population is exactly compensated by its fourfold lower model-side cost per trajectory. Agentic PPO is capped at 500 training steps to remain within a comparable compute budget. For the measured values in Table 2, we further replace the nominal trajectory length by the actual number of model-processed tokens, yielding 3.1/6.3/9.4 EFLOPs for Agentic ESOpt and 3.2/7.6/10.9 EFLOPs for Agentic GRPO at $H^* \in \{5, 10, 15\}$. The extra FLOPs of Agentic GRPO are because the algorithm’s drawback in long-horizon credit assignment often results in longer trajectories.

Math Reasoning and DocVQA. For Math Reasoning, Agentic ESOpt ($G = 16$) and Agentic GRPO ($G = 8$) each cover the 400-example training set once. For DocVQA, both methods likewise cover 640 training examples. Under comparable trajectory lengths, their relative model-side FLOPs are therefore

$$\frac{\text{FLOPs}_{\text{ES}}}{\text{FLOPs}_{\text{GRPO}}} \approx \frac{2 \times 16}{8 \times 8} = \frac{1}{2}.$$

Agentic ESOpt therefore requires approximately half the model-side training FLOPs of the matched Agentic GRPO configuration on these two tasks.

D Experimental Settings and Additional Results

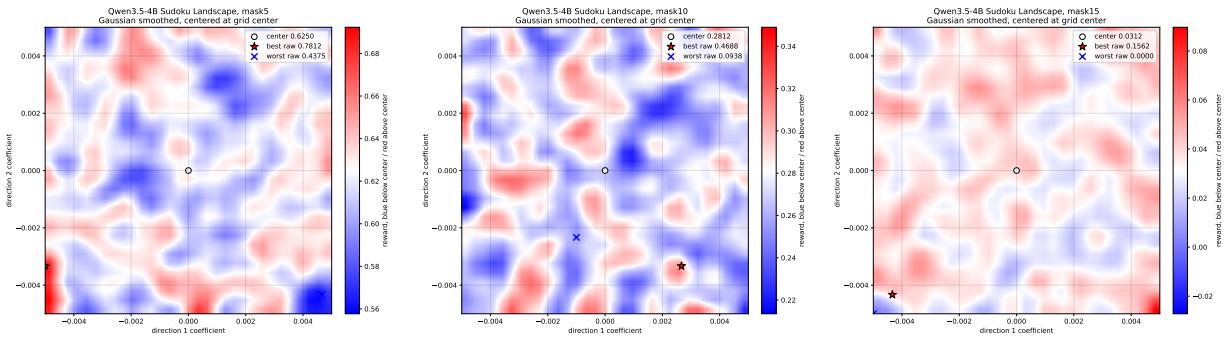
This appendix provides the task-specific implementation details and additional diagnostics supporting the experiments in Sections 4–6. We follow the order of the main experiments: Sudoku, Math reasoning, DocVQA, WebArena-Lite, and automatic heuristic design, followed by a cross-setting summary of Agentic ESOpt hyperparameters.

D.1 Sudoku

D.1.1 Empirical Diagnostics for Long-Horizon Scaling

To complement the long-horizon scaling analysis in Section 4 and Appendix C.3, we examine three empirical diagnostics on Sudoku as the minimum successful horizon H^* increases. We first visualize the local parameter-space reward landscape around the same Qwen3.5-4B checkpoint. We then relate per-turn correctness to trajectory-level success through a simple schematic calculation, and finally compare the realized horizons of Agentic GRPO and Agentic ESOpt during training.

For the landscape visualization, we select two orthogonal directions in the d -dimensional parameter space and evaluate a two-dimensional plane spanned by these directions around the current checkpoint. Each point in the heatmap corresponds to a perturbed model in that slice, and its value is the Gaussian-smoothed average terminal reward measured on the Sudoku batch. Since Sudoku provides only binary terminal feedback, this value can be interpreted as a smoothed local success score. The theoretical point is not that longer horizons leave the reward landscape unchanged, but that ES does not introduce an additional explicit accumulation over turns in its parameter-score factor.



(a) Minimum successful horizon $H^* = 5$ (b) Minimum successful horizon $H^* = 10$ (c) Minimum successful horizon $H^* = 15$

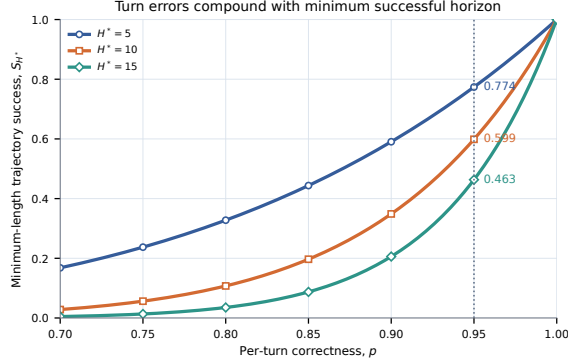
Figure 5 Two-dimensional Gaussian-smoothed reward landscapes around the Qwen3.5-4B checkpoint for Sudoku tasks with $H^* \in \{5, 10, 15\}$. Each panel marks the unperturbed checkpoint and the best and worst raw perturbations in the displayed slice. The heatmap value is the smoothed average terminal reward in the local neighborhood. As H^* increases, the absolute reward level and contrast decrease, reflecting increasingly sparse trajectory feedback, while local parameter-space variation remains observable.

As shown in Figure 5, increasing H^* substantially reduces the absolute reward level and reward contrast, making informative terminal feedback increasingly scarce. Nevertheless, useful variation remains visible in the local parameter neighborhood even at $H^* = 15$. This is consistent with our theoretical characterization: longer horizons still make the return signal harder, but Agentic ESOpt does not incur an additional horizon-wise accumulation in its parameter-score factor.

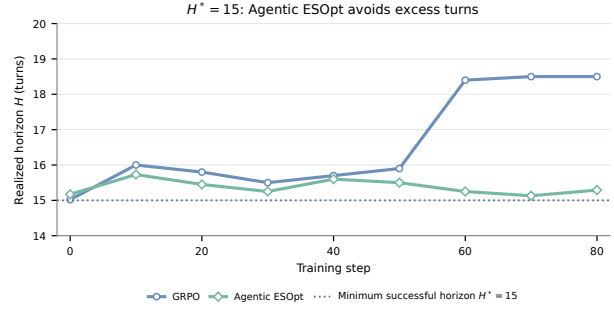
Following the Sudoku step-accuracy definition of Kim et al. (2026), a turn is correct when it fills a masked cell with its unique target value. To connect per-turn correctness with trajectory-level success, consider a minimum-length successful path with homogeneous per-turn correctness probability p . Its success probability is

$$S_{H^*} = p^{H^*}.$$

This relation is schematic and is not fitted to the measured Sudoku trajectories; it simply illustrates how residual turn errors compound with the number of required actions.



(a) Schematic compounding from per-turn correctness to trajectory success.



(b) Measured realized-horizon diagnostics on the $H^* = 15$ setting.

Figure 6 Turn-level diagnostics for long-horizon Sudoku. Left: under the schematic relation $S_{H^*} = p^{H^*}$, the same per-turn error rate produces a larger trajectory-level penalty as the minimum successful horizon grows. For example, $p = 0.95$ gives $S_{H^*} = 0.774$, 0.599 , and 0.463 for $H^* = 5, 10$, and 15 , respectively. Right: realized-horizon diagnostics on the $H^* = 15$ setting. Agentic GRPO training uses temperature 1, top- $p = 1$, and top- $k = -1$, while its evaluation uses temperature 0.7, top- $p = 0.8$, and top- $k = 20$; Agentic ESOpt uses the latter decoder for both training and evaluation. Agentic GRPO reaches the 45-turn interaction budget by step 60, whereas Agentic ESOpt remains near the 15-turn minimum and ends at 15.41 turns.

Figure 6 connects the theoretical argument to observable behavior. The left panel shows that even small residual turn errors become much more destructive as the required horizon grows. The right panel shows the corresponding training-time separation: Agentic GRPO progressively accumulates excess turns, whereas Agentic ESOpt stays close to the minimum-length solution path. These diagnostics are not a complete causal proof, but they are consistent with the estimator-level distinction in Section 4: Agentic ESOpt evaluates one coherent parameter perturbation using the complete trajectory outcome, rather than propagating terminal feedback through an expanding sequence of action-level decisions.

D.1.2 Training Configuration

Agentic ESOpt and Vanilla ES. All fixed ES comparisons use Qwen3.5-4B for 100 generations. At each generation, $G = 32$ full-parameter directions are evaluated on the same batch of 32 puzzles, and the resulting trajectory rewards are z-score normalized before an update with $\alpha = 5 \times 10^{-4}$. The interaction budget is three times the mask count—15, 30, and 45 turns for $H^* = 5, 10$, and 15 —with at most 64 generated tokens per turn. Training and evaluation use temperature 0.7, top- $p = 0.8$, top- $k = 20$, min- $p = 0$, presence penalty 1.5, and repetition penalty 1.0. Evaluation is performed every 10 generations with three repeated runs.

Table 8 Sudoku ES configurations. All profiles use $G = 32$, 32 puzzles per generation, 100 generations, and $\alpha = 5 \times 10^{-4}$.

Profile	Mask count	Sigma schedule	Maximum turns
Vanilla ES	5 / 10	1.0×10^{-3} constant	15 / 30
Vanilla ES	15	5.0×10^{-4} constant	45
Agentic ESOpt	5 / 10	$1.0 \times 10^{-3} \rightarrow 2.5 \times 10^{-4}$ cosine	15 / 30
Agentic ESOpt	15	$7.0 \times 10^{-4} \rightarrow 5.0 \times 10^{-4}$ cosine	45

Agentic RL. Both Agentic GRPO profiles run 100 rollout-update rounds with a global prompt batch of 32 and eight rollouts per prompt. They use rollout micro-batch 8, policy-training micro-batch 2, learning rate 10^{-6} , KL coefficient 10^{-3} , and clipping coefficient 0.2. GRPO-A samples with temperature 0.7, top- $p = 0.8$, and top- $k = 20$, while GRPO-B samples with temperature 1, top- $p = 1$, and top- $k = -1$. Both are evaluated with the Agentic ESOpt decoder before training and every 20 rounds, with three repeated evaluations. One round denotes a complete rollout batch followed by the corresponding policy update.

We implement Turn-PPO (Li et al., 2026) as the Agentic PPO baseline. Turn-PPO uses a separate Qwen3.5-4B critic initialized with a newly added value head for turn-level advantage estimation. We follow the same

interaction and evaluation protocol as the other Agentic RL baselines (temperature 1, top- $p = 1$, and top- $k = -1$ in training and with temperature 0.7, top- $p = 0.8$, and top- $k = 20$ for evaluation), and train at most 500 steps to align the FLOPs.

D.1.3 Vanilla-ES Population-Sensitivity Configuration

The 4B/9B comparison in Table 6 uses the $H^* = 15$ Sudoku environment for 100 generations and evaluates $G \in \{8, 16\}$ every 10 generations. To isolate population sensitivity, all runs use Vanilla ES with a constant perturbation scale $\sigma = 5 \times 10^{-4}$, update scale $\alpha = 5 \times 10^{-4}$, full-parameter perturbations, population z-score normalization, and eight training puzzles per direction. Each generation therefore contains $8G$ direction-puzzle rollouts. Perturbation rollouts and evaluation use temperature 0.7, top- $p = 0.8$, and top- $k = 20$, and each evaluation is repeated three times.

The environment provides a binary terminal reward: a rollout receives 1 only when the final board is a complete legal Sudoku solution preserving all given cells. Invalid formats, out-of-range values, attempts to modify given cells, and overwrites leave the board unchanged. This fixed setup makes population size the only varied ES hyperparameter in the ablation; the cosine perturbation schedule and intermediate reward shaping are not used.

D.2 Math Reasoning

All method-specific configurations described below refer to training unless otherwise stated. Final evaluation is conducted under a common protocol shared by Agentic ESOpt, Agentic GRPO, Trace2Skill, their combined variants, and all other reported baselines. For Trace2Skill, we sample 16 runs for each instance and follow the instructions from Ni et al. (2026) in using at most one error trajectory per instance to distill skill.

D.2.1 Agentic ESOpt Training Configuration

Experiments of ReAct-style Math Reasoning are conducted on four A100 80GB GPUs. We first optimize the No Skill Qwen3.5-4B policy and record the seed of every parameter update together with the trajectories later supplied to Trace2Skill. For final evaluation, the base checkpoint is reloaded, and the update sequence is deterministically reconstructed. The No Skill and skill-conditioned evaluations therefore share an identical parameter stage and differ only in the external skill context.

Table 9 Agentic ESOpt training configuration for Math reasoning.

Parameter	Value
Backbone	Qwen3.5-4B
ES generations	25
Population size G	16
Cases per direction	16
Update scale α	5×10^{-4}
σ schedule	$10^{-3} \rightarrow 5 \times 10^{-4}$ cosine
Reward normalization	Population z-score
Maximum turns	50
Training generation limit	4096 tokens / turn

Each direction is evaluated on 16 problems, giving $16 \times 16 = 256$ direction-problem trajectories per generation. Training uses temperature 1, top- $p = 1$, top- $k = 40$, min- $p = 0$, presence penalty 2, and repetition penalty 1. The trajectory reward is exact match on the parsed final answer. Intermediate evaluation is performed every 10 generations with one sample per problem.

D.2.2 Multi-Turn GRPO Training Configuration

The Math Agentic GRPO baseline follows the same sampling configuration as Agentic ESOpt, with learning rate 10^{-6} , eight rollouts per prompt, KL coefficient 10^{-3} using the low-variance KL form, temperature 1, top- $p = 1$, top- $k = 40$, presence penalty 2, and repetition penalty 1.

For full-parameter Agentic GRPO training, we appropriately compress the 4096-token per-turn generation configuration to satisfy the training-memory constraint on four A100 80GB GPUs. This adjustment applies only to the training rollout configuration and does not affect the final evaluation protocol. The 400 training records form 20 prompt batches per epoch.

D.2.3 Common Evaluation Configuration

All reported methods and baselines use the same final evaluation protocol. For Math reasoning, we evaluate on 100 held-out DAPO problems and 30 AIME 2026 problems, with four samples per problem. Each trajectory is allowed at most 50 interaction turns and up to 4096 generated tokens per turn. We use temperature 1, top- $p = 1$,

Table 10 Multi-turn Agentic GRPO training configuration for Math reasoning.

Parameter	Value
Backbone	Qwen3.5-4B
Training records	400
Prompt batch size	20
Rollouts per prompt	8
Training epochs	1
Training rounds	20
Total trajectories	3,200
Learning rate	10^{-6}
KL coefficient	10^{-3}
Maximum interaction	100 user + 100 assistant turns

top- $k = 40$, min- $p = 0$, presence penalty 2, and repetition penalty 1 for all methods. These evaluation settings are shared by Agentic ESOpt, Agentic GRPO, Trace2Skill, their combined variants, and all other reported baselines, independent of their method-specific training configurations.

D.3 DocVQA

As in Math reasoning, the configurations below specify the corresponding training procedures unless otherwise stated. For DocVQA, Agentic ESOpt and Agentic GRPO additionally use fully aligned interaction and generation settings during training, and all reported methods share the same final evaluation configuration. For Trace2Skill, we sample 16 runs for each instance and follow the instructions from Ni et al. (2026) in using at most one error trajectory and one correct trajectory per instance to distill skill.

D.3.1 Agentic ESOpt Training Configuration

Experiments of ReAct-style DocVQA are also conducted on four A100 80GB GPUs. The DocVQA experiment follows the same paired reconstruction protocol as Math: the No Skill run records update seeds and source trajectories, and final evaluation replays the identical parameter sequence before adding the post-hoc skill context.

Table 11 Agentic ESOpt training configuration for DocVQA.

Parameter	Value
Backbone	Qwen3.5-4B
ES generations	40
Population size G	16
Cases per direction	16
Update scale α	5×10^{-4}
σ schedule	$10^{-3} \rightarrow 5 \times 10^{-4}$ cosine
Reward normalization	Population z-score
Maximum turns	50
Generation limit	512 tokens / turn
Total generation cap	32,768 tokens / trajectory

Each generation contains $16 \times 16 = 256$ direction-question trajectories. Training uses temperature 1, top- $p = 1$, top- $k = 40$, min- $p = 0$, presence penalty 2, and repetition penalty 1. A valid trajectory must contain at least one parsed tool action before the final answer. Continuous ANLS is used as the training reward; threshold accuracy counts an answer as correct when ANLS is strictly greater than 0.5. Intermediate evaluation is performed every 10 generations with one sample per question.

D.3.2 Multi-Turn GRPO Training Configuration

DocVQA Agentic GRPO uses the same sampling configuration and interaction budget as Agentic ESOpt, together with the same optimizer and KL settings as the Math baseline.

Agentic GRPO and Agentic ESOpt therefore use the same 50-turn interaction limit, 512-token per-turn generation limit, total trajectory generation cap, and sampling configuration during training. The final two records do not fill a four-question batch and are omitted, leaving 12 training rounds per epoch. Continuous ANLS is used as the training reward as well.

D.3.3 Common Evaluation Configuration

All reported methods and baselines also follow the same final evaluation protocol on DocVQA. We evaluate on the same held-out set of 100 questions with four samples per question. Each trajectory is limited to 50 interaction

Table 12 Multi-turn Agentic GRPO training configuration for DocVQA.

Parameter	Value
Backbone	Qwen3.5-4B
Training records	50
Prompt batch size	4
Rollouts per prompt	8
Training epochs	15
Training rounds	180
Total trajectories	5,760
Learning rate	10^{-6}
KL coefficient	10^{-3}
Maximum turns	50
Generation limit	512 tokens / assistant turn
Total generation cap	32,768 tokens / trajectory

turns, 512 generated tokens per turn, and 32,768 generated tokens in total. Sampling uses temperature 1, top- $p = 1$, top- $k = 40$, min- $p = 0$, presence penalty 2, and repetition penalty 1 for every method. Agentic ESOpt, Agentic GRPO, Trace2Skill, their combined variants, and all other reported baselines therefore use identical evaluation settings. In addition, the interaction and generation budgets of Agentic ESOpt and Agentic GRPO are fully aligned during DocVQA training.

D.3.4 Repeated-Sampling Profiles for Math and DocVQA

Figure 7 extends the four-sample results in Table 3 to repeated-sampling budgets up to $k = 32$, using the same evaluation decoder for each method.

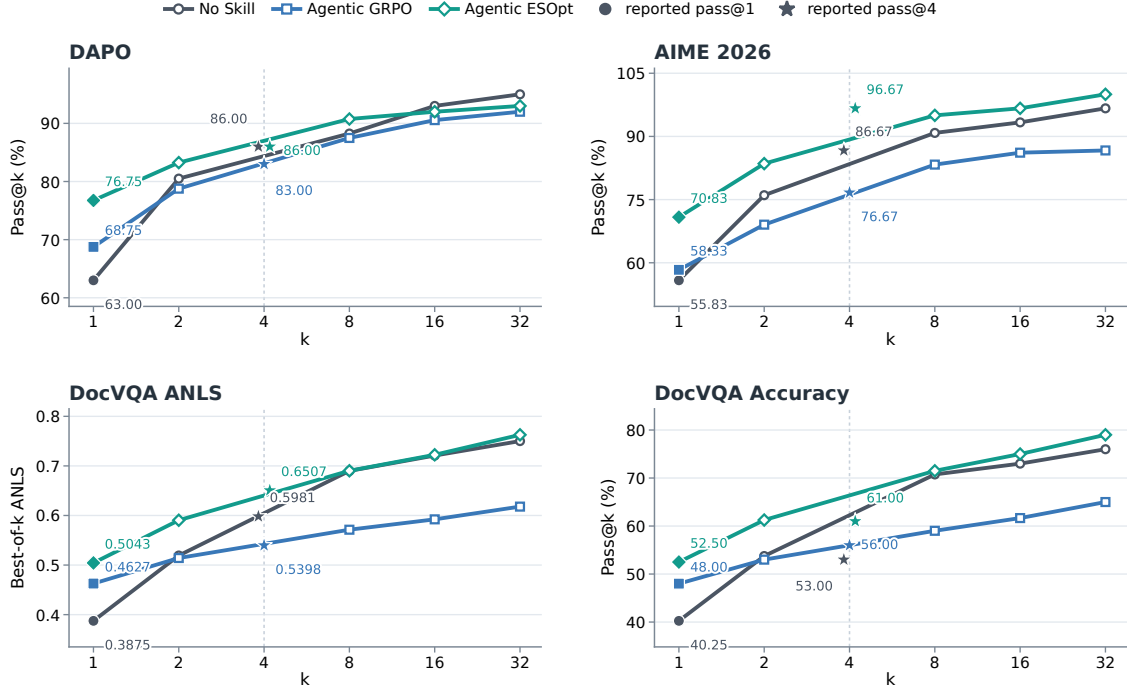


Figure 7 Repeated-sampling profiles for DAPO, AIME 2026, DocVQA ANLS, and DocVQA accuracy. The $k = 1$ markers use the reported Mean@4 values as estimates of single-sample performance, while stars show the originally reported Pass@4 or Max@4 values. Curves report the newly estimated best-of- k results for $k \in \{2, 8, 16, 32\}$; the newly computed $k = 4$ curve points are omitted to keep the original four-sample measurements visually distinct.

Across all four panels, Agentic ESOpt remains above the matched Agentic GRPO baseline as the sampling budget increases. On AIME 2026 and both DocVQA metrics, it also preserves an advantage over the base policy at larger k , indicating that the average-performance gains are not obtained by collapsing repeated-sampling coverage.

D.3.5 DocVQA Training-Stage Turn Diagnostics

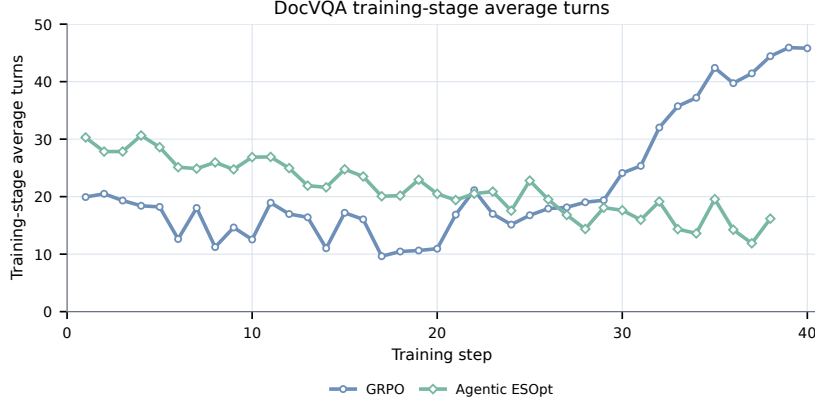


Figure 8 Average realized trajectory length during DocVQA training. Agentic GRPO and Agentic ESOpt use the same rollout decoder; their training schedules are aligned to a common update axis and shown through step 40.

The two methods begin with similar trajectory lengths, but their behavior diverges later in training: Agentic GRPO trajectories grow rapidly, while Agentic ESOpt remains shorter and more stable. This diagnostic mirrors the excess-turn pattern observed in Sudoku and provides an additional view of optimization under sparse trajectory-level feedback.

D.4 WebArena-Lite

D.4.1 Goal-Conditioned Task Formulation

WebArena-Lite evaluates goal-conditioned control in a partially observed browser environment. A task g specifies a user goal and an initial latent web state. At time t , the agent receives observation o_t , samples an action

$$a_t \sim \pi_\theta(\cdot \mid o_{\leq t}, a_{< t}, g, c),$$

and transitions to the next browser state. A trajectory $\tau = (o_0, a_0, \dots, o_T)$ is successful when it satisfies the goal predicate within the interaction budget. The benchmark objective is

$$J_{\text{web}}(\theta; c) = \mathbb{E}_{g \sim \mathcal{G}} \mathbb{E}_{\tau \sim \pi_\theta(\cdot \mid g, c)} [\mathbb{1}_{\text{succ}}(\tau, g)],$$

estimated by the empirical success rate over the held-out task set. The reward is sparse: intermediate actions receive no direct task credit, and the final score depends on whether the complete trajectory reaches the target web state.

The external context c represents non-parametric adaptation. In the No Skill condition, it contains the task instruction and interaction history; in the Trace2Skill condition, it additionally contains procedural knowledge distilled from previous trajectories. The No Skill pair in Table 4 isolates the parameter update, while the skill-conditioned pair compares the complete adaptation pipelines.

For website category $\mathcal{G}_k$, the reported conditional success rate is

$$\hat{J}_{\mathcal{G}_k}(\theta; c) = \frac{1}{|\mathcal{G}_k|} \sum_{g \in \mathcal{G}_k} \mathbb{1}_{\text{succ}}(\tau_g, g).$$

The dataset average applies the same estimator to the full set $\mathcal{G}$.

D.4.2 Rollout Protocol and Data Split

All frozen and Agentic ESOpt rows use the same browser protocol. The agent observes WebRL-style textual browser states, acts in the WebRL id-based action space, and is limited to 30 browser actions per task. Each response has a 2,048-token budget, the browser viewport is 1280×720 , and the evaluator assigns binary success after termination. Sampling uses temperature 0.7, top- $p = 0.8$, top- $k = 20$, min- $p = 0$, presence penalty 1.5, and repetition penalty 1. Final results average three complete evaluations over all 165 held-out tasks.

The training split is constructed from the original 812 WebArena tasks after excluding the 165 tasks mapped to WebArena-Lite by the benchmark-provided correspondence. Site-stratified splitting of the remaining 647

tasks yields 582 training tasks and 65 validation tasks. We run each model with three seeds to distill the skill (base model, Agentic RL model, Agentic ESOpt model) and select the best one on the validation dataset as the evaluation skill. WebArena-Lite tasks never contribute parameter-update rewards or trajectory-to-skill inputs, and periodic evaluation is read-only. All six sites—Reddit, GitLab, Wikipedia, Map, Shopping, and Shopping Admin—remain enabled.

D.4.3 Parameter and Skill Adaptation Stages

Table 13 WebArena-Lite adaptation stages on Qwen3.5-27B. Both Agentic ESOpt conditions share the same No Skill parameter updates; the combined condition adds one post-hoc skill-distillation stage.

Condition	Parameter stage	Skill stage	Data per stage	Additional ES
No Skill baseline	none	none	—	none
Agentic ESOpt + No Skill	70 ES generations	none	$G = 8 \text{ directions} \times 8 \text{ tasks/generation}$	—
Trace2Skill baseline	none	70 skill iterations	$8 \text{ tasks} \times 8 \text{ rollouts/iteration}$	none
Agentic ESOpt + Trace2Skill	same 70-generation ES run	one post-hoc distillation	all completed ES trajectories	none

The Trace2Skill baseline starts from an empty skill and runs 70 skill-evolution iterations. Each iteration uses eight training tasks with eight sampled trajectories per task and selects at most one positive and one negative trajectory per task. Validation is performed every 10 iterations, and Table 4 reports the final evaluated skill.

The No Skill parameter stage performs 70 full-parameter ES updates with $G = 8$ on eight training tasks per generation. Each generation therefore contains $8 \times 8 = 64$ direction–task evaluations, with every perturbation scored on the same task batch. Rewards are z-score normalized, $\sigma_0 = 1.5 \times 10^{-3}$, $\sigma_t = 1.5 \times 10^{-3}$, $\alpha = 2.5 \times 10^{-4}$, and evaluation is performed every 10 generations.

After the parameter stage, trajectories from all completed ES generations are passed once to Trace2Skill. Final evaluation reconstructs the same No Skill update sequence and adds the distilled skill to the system prompt. This experiment implements sequential shared-trajectory composition; no additional skill-conditioned ES stage is performed.

D.4.4 Full-Evaluation Curve

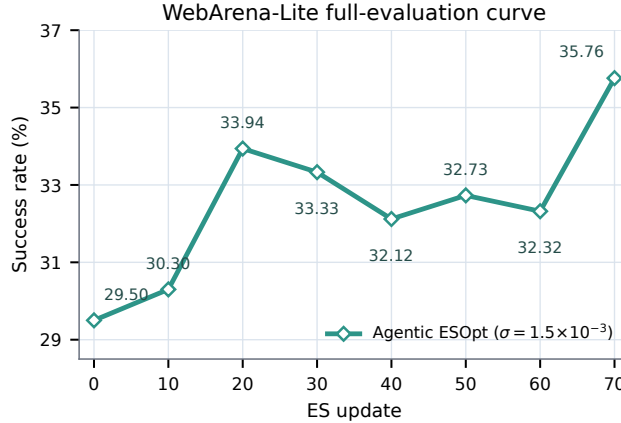


Figure 9 Full WebArena-Lite evaluation success rate during the No Skill Qwen3.5-27B Agentic ESOpt run. Evaluation is performed every 10 ES updates; the curve starts from the 29.50% base checkpoint and reaches 35.76% after 70 updates.

The full-set curve is non-monotonic across intermediate checkpoints, but the final update reaches the best observed success rate. Table 4 reports the three-run final evaluation used for the main comparison.

D.5 Automatic Heuristic Design

D.5.1 Experimental Configuration

Experiments of AHD are done on a cluster of eight 3090 24GB GPUs. We evaluate Agentic ESOpt under two fixed outer search scaffolds: independent Sample and EoH. In each paired comparison, the proposal budget and

outer search procedure are unchanged; Agentic ESOpt adds only the parameter-space update. All runs use LLaMA-3.1-8B-Instruct.

EoH maintains $N = 10$ heuristics for 25 outer generations. Operators e1 and e2 each generate N candidates, while m1 and m2 each generate kN candidates, giving a total budget of $25N(2 + 2k)$. We use $k = 1$ for $T = 1,000$ and $k = 3$ for $T = 2,000$. The Sample scaffold evaluates independent i1 proposals in batches of 20.

Table 14 AHD proposal budgets and Agentic ESOpt update units. Matched baselines use the same proposal counts. In EoH, m1 and m2 each form one ES update batch per outer generation.

Outer scaffold	T	Outer generations	Candidates/generation	ES update batches	Directions/batch	$\sigma_0 \rightarrow \sigma_T$
EoH ($k = 1$)	1,000	25	$10(2 + 2k) = 40$	50	10	$10^{-3} \rightarrow 0$
EoH ($k = 3$)	2,000	25	$10(2 + 2k) = 80$	50	30	$10^{-3} \rightarrow 0$
Independent Sample	1,000	50	20	50	20	$10^{-3} \rightarrow 0$
Independent Sample	2,000	100	20	100	20	$10^{-3} \rightarrow 0$

Agentic ESOpt uses full-parameter, one-sided Gaussian perturbations, cosine decay $\sigma : 10^{-3} \rightarrow 0$, no warmup, $\alpha = 5 \times 10^{-4}$, and population z-score normalization. In EoH, only mutation operators m1 and m2 trigger parameter updates, yielding two update batches per outer generation; e1 and e2 remain unchanged. In Sample, each batch of 20 proposals forms one update.

All objectives are converted internally to minimization costs. EoH uses the selected parent’s cost minus the perturbed child’s cost, while Sample uses the negative child score. Invalid or non-finite programs receive a finite within-batch penalty before normalization, and an all-invalid batch produces no update. Candidate generation uses temperature 1, top- $p = 0.98$, no top- k cutoff, and at most 768 new tokens.

D.5.2 Additional ACO-Style Results and Ablations

Table 15 AHD ACO-style results at total evaluation budgets $T \in \{1000, 2000\}$. Lower is better for all columns. The Δ rows report the ratio $\text{EoH}/(\text{Agentic ESOpt} + \text{EoH}) - 1$; positive ratios denote improvement. Improvements are shaded green with green deltas; regressions remain unshaded with neutral-gray deltas.

Method	TSP $N = 50$	TSP $N = 100$	CVRP $N = 50, C = 50$	CVRP $N = 100, C = 50$	BPP $N = 500, C = 150$	BPP $N = 1,000, C = 150$
ACO	5.992	8.948	11.355	18.778	208.828	417.938
<i>Total Evaluations: $T = 1000$</i>						
EoH	6.030	8.859	9.328	15.666	203.162	405.172
Agentic ESOpt + EoH	5.882	8.391	9.265	16.073	203.500	405.787
Δ vs EoH	↓2.53%	↓5.57%	↓0.68%	↑2.53%	↑0.17%	↑0.15%
<i>Total Evaluations: $T = 2000$</i>						
EoH	5.886	8.368	9.379	15.989	203.307	405.688
Agentic ESOpt + EoH	5.890	8.398	9.179	15.420	203.052	405.276
Δ vs EoH	↑0.08%	↑0.36%	↓2.18%	↓3.69%	↓0.13%	↓0.10%

ACO-style results. At $T = 1,000$, Agentic ESOpt + EoH improves three of the six ACO-style test sets: both TSP settings and CVRP-50. At $T = 2,000$, it improves both CVRP and both BPP settings; the two TSP differences are small (0.08% and 0.36%). Together with the constructive Sample and EoH comparisons, Agentic ESOpt improves 28 of 36 matched method–budget settings across 12 test sets and six scenarios, with one tie and seven regressions.

Table 16 Component ablation on constructive AHD at $T = 1000$. TSP is minimized, and KP is maximized.

Method	TSP	KP
	$N = 50$	$N = 50, W = 12.5$
EoH (baseline)	6.545	19.996
Agentic ESOpt + EoH	6.463	20.001
w/o ES (noise-only)	6.484	19.996
w/o cosine schedule	6.480	19.997

Table 17 EoH sampling-temperature ablation on constructive TSP ($N = 50$) at $T = 1000$, with Agentic ESOpt included for comparison. Lower is better.

Setting	TSP
EoH (0.6)	6.51782
EoH (1.0)	6.545
EoH (1.5)	6.95927
Agentic ESOpt	6.463

Component and sampling-temperature ablations. The component ablation in Table 16 shows that both the reward-weighted parameter update and the cosine perturbation schedule contribute to the final result: retaining noise without the ES update, or fixing the perturbation radius, weakens performance on both tasks. Table 17 further shows that the gain is not reproduced by retuning the EoH sampling temperature. Temperature 0.6 is the strongest EoH setting, yet Agentic ESOpt remains better on TSP.

Repeated-run significance. We repeat the constructive EoH comparison 20 times per method on TSP ($N = 50$) and KP ($N = 100, W = 25$). Table 18 reports the mean and sample standard deviation across runs together with a one-sided, equal-variance t -test. KP values are converted back from the internally negated solver cost to the native maximization objective. The test is computed from the same 20 per-run observations summarized in the table.

Table 18 Repeated-run analysis for constructive AHD. Each method is evaluated over 20 independent runs. Lower is better for TSP and higher is better for the native KP objective.

Task	EoH	Agentic ESOpt + EoH	p -value
TSP ($N = 50, \downarrow$)	6.5517 ± 0.0729	6.5007 ± 0.0868	0.0258
KP ($N = 100, W = 25, \uparrow$)	40.1562 ± 0.0024	40.1578 ± 0.0017	0.0100

Both comparisons are significant at the 0.05 level. The repeated-run analysis therefore supports the consistency of the AHD gains beyond a single search seed.

On-the-fly runtime overhead. We additionally measure end-to-end wall-clock time for the constructive Sample scaffold at $T = 1000$.

Table 19 End-to-end runtime for constructive Sample at $T = 1000$. All values are in minutes.

Task	Sample	Agentic ESOpt + Sample
Design constructive heuristics for TSP	54.8	60.6
Design constructive heuristics for KP	41.2	48.6
Design constructive heuristics for ASP	51.7	56.7

Across the three tasks, attaching Agentic ESOpt adds only 5.0–7.4 minutes, corresponding to a 9.7%–18.0% increase over the original Sample runtime. The parameter updates are therefore carried out nearly on the fly within the existing candidate-generation and evaluation loop, without a separate post-search training stage.

D.6 Cross-Setting Hyperparameter Summary

Table 20 consolidates the Agentic ESOpt configurations used across all reported settings. The task-specific sections above provide the corresponding interaction limits, baseline settings, and evaluation protocols.

Table 20 Cross-setting Agentic ESOpt configurations. “Update batches” counts parameter updates, and “reward load/direction” specifies how each perturbation is scored. All settings use full-parameter, one-sided Gaussian perturbations and population z-score normalization.

Setting	Backbone	Update batches	Directions/update	Reward load/direction	$\sigma_0 \rightarrow \sigma_T$	α
Sudoku, $H^* = 5/10$	Qwen3.5-4B	100	32	32 puzzles	$10^{-3} \rightarrow 2.5 \times 10^{-4}$	5×10^{-4}
Sudoku, $H^* = 15$	Qwen3.5-4B	100	32	32 puzzles	$7 \times 10^{-4} \rightarrow 5 \times 10^{-4}$	5×10^{-4}
Math	Qwen3.5-4B	25	16	16 problems	$10^{-3} \rightarrow 5 \times 10^{-4}$	5×10^{-4}
DocVQA	Qwen3.5-4B	40	16	16 examples	$10^{-3} \rightarrow 5 \times 10^{-4}$	5×10^{-4}
WebArena-Lite	Qwen3.5-27B	70	8	8 web tasks	$1.5 \times 10^{-3} \rightarrow 1.5 \times 10^{-3}$ constant	2.5×10^{-4}
AHD + EoH, $T = 1,000$	Llama-3.1-8B	50	10	one heuristic objective	$10^{-3} \rightarrow 0$	5×10^{-4}
AHD + EoH, $T = 2,000$	Llama-3.1-8B	50	30	one heuristic objective	$10^{-3} \rightarrow 0$	5×10^{-4}
AHD + Sample, $T = 1,000/2,000$	Llama-3.1-8B	50 / 100	20	one heuristic objective	$10^{-3} \rightarrow 0$	5×10^{-4}

For candidate i , the implementation applies $\theta + \sigma_t \epsilon_i$, evaluates the complete trajectory or generated heuristic, reverts the perturbation, and reconstructs the same direction from its integer seed during the update. The implementation uses one-sided perturbations and no explicit $1/\sigma_t$ multiplier, so α is the effective update scale. Task-specific baseline configurations are summarized in [Tables 8 to 14](#).

The perturbation schedule reflects the optimization objective of each setting. Train-time experiments retain a nonzero terminal radius to preserve neighborhood smoothing: Sudoku uses horizon-specific terminal radii, Math and DocVQA decay to 5×10^{-4} , and WebArena-Lite keeps $\sigma = 1.5 \times 10^{-3}$. AHD decays σ to zero because the final objective is the best task-specific heuristic found during test-time search.

Population size G and the number of tasks used to score each direction are chosen jointly. Increasing either quantity can improve update stability or task coverage, while also increasing rollout cost. Across settings, binary-reward tasks use multiple tasks per direction, whereas AHD assigns one executable heuristic objective to each direction.

Population z-score normalization provides a common update interface across heterogeneous rewards. It converts binary success, continuous ANLS, and problem-dependent heuristic objectives into relative within-population scores. For AHD, invalid or non-finite programs are mapped to finite within-batch penalties before normalization, and an all-invalid batch produces no parameter update.

Hyperparameter sensitivity and adaptive schedules for G and σ are discussed in [Appendix A](#).

E Prompts and Skills

E.1 Sudoku

E.1.1 Environment Prompt

Sudoku is implemented as a multi-turn action environment. The agent does not submit a full board in one response. Instead, at each turn it observes the original puzzle, the current board, the current empty-cell list, and optional environment feedback from the previous action, then emits exactly one action in the format **set** **<row>** **<col>** **<value>**. Rows and columns are 1-indexed. The episode terminates when all masked cells are filled, or the action budget is exhausted, and the final reward is binary.

The following is a turn-0 prompt from the evaluation split with 15 masked cells. Empty cells are rendered as “.”.

Prompt for Sudoku

```
You are an agent solving Sudoku one action at a time. At each turn, fill exactly one empty cell. Rows and columns are 1-indexed. The board is split by | and horizontal lines into nine 3x3 boxes. Every row, every column, and every 3x3 box must contain digits 1 through 9 exactly once. Choose the cell only from the Current empty cells list. Keep all original givens fixed. Your entire response must be exactly one line containing one action and nothing else. Do not explain, reason aloud, wrap the answer in code fences, or output any extra words. Use exactly this format:
set <row> <col> <value>

Original puzzle, mask_count=15:
  c1 c2 c3 | c4 c5 c6 | c7 c8 c9
r1: 3 5 9 | 4 . 7 | . . 8
r2: 2 1 8 | . 9 5 | 4 . 6
r3: 4 7 6 | 2 8 1 | 3 5 9
-----+-----+-----
r4: 6 2 1 | 8 5 3 | 9 4 7
r5: . 3 5 | 9 . 4 | 6 2 1
r6: . 4 7 | 6 1 2 | 8 3 5
-----+-----+-----
r7: 7 6 . | 1 3 8 | 5 9 .
r8: 5 9 . | . 2 6 | 1 . 3
r9: . 8 3 | 5 4 . | 7 6 2

Current board, turn=0, remaining_empty=15:
  c1 c2 c3 | c4 c5 c6 | c7 c8 c9
r1: 3 5 9 | 4 . 7 | . . 8
r2: 2 1 8 | . 9 5 | 4 . 6
r3: 4 7 6 | 2 8 1 | 3 5 9
-----+-----+-----
r4: 6 2 1 | 8 5 3 | 9 4 7
r5: . 3 5 | 9 . 4 | 6 2 1
r6: . 4 7 | 6 1 2 | 8 3 5
-----+-----+-----
r7: 7 6 . | 1 3 8 | 5 9 .
r8: 5 9 . | . 2 6 | 1 . 3
r9: . 8 3 | 5 4 . | 7 6 2

Current empty cells: r1c5, r1c7, r1c8, r2c4, r2c8, r5c1, r5c5, r6c1, r7c3, r7c9, r8c3, r8c4, r8c8, r9c1, r9c6
```

For example, a valid response to this prompt is a single line such as **set 1 5 6**. The environment then applies the action and constructs the next prompt by inserting **Last environment feedback: Filled r1c5 with 6.** before the original puzzle block, updating the current board, setting **turn=1**, decreasing **remaining_empty** to 14, and removing r1c5 from the current empty-cell list.

E.2 Math Reasoning

E.2.1 ReAct Prompt

Instead of a single direct answer call, the AIME setting is treated as an agentic ReAct environment. The model may call a command-line Python tool multiple times, receives each tool result as an observation, and stops when it emits the final boxed answer. The ES reward is the binary exact-match score of the final answer after the whole trajectory.

Prompt for AIME ReAct

```
You are solving a math reasoning problem. You can think step-by-step and use command-line Python when useful.

Action format:
Action:
{
  "name": "bash",
  "arguments": {"command": "<shell command>"}
}

After each Action, the environment executes the command and returns an Observation. Repeat Thought/Action/Observation as needed. When finished, output the final result as \boxed{answer}.

Problem:
Patrick started walking at a constant rate along a straight road from school to the park. One hour after Patrick left, Tanya started running along the same road from school to the park. One hour after Tanya left, Jose started bicycling along the same road from school to the park. Tanya ran
```

at a constant rate of 2 miles per hour faster than Patrick walked, Jose bicycled at a constant rate of 7 miles per hour faster than Tanya ran, and all three arrived at the park at the same time. The distance from the school to the park is m/n miles, where m and n are relatively prime positive integers. Find $m+n$.

Thought: Let Patrick's travel time be T and speed be p . I can solve the two equal-distance equations with Python.

Action:

```
{
  "name": "bash",
  "arguments": {"command": "python -c \"import sympy as sp; T,p=sp.symbols('T p'); sol=sp.solve([sp.Eq(p,2*T-2), sp.Eq(p,sp.Rational(9,2)*T-9)],[T,p]); T0,p0=sol[T],sol[p]; d=sp.factor(T0*p0); print(d, sp.numer(d)+sp.denom(d))\""}
}
```

Observation: 252/25 277

Thought: The distance is 252/25, so $m+n=277$.

Final answer: $\boxed{277}$

E.2.2 Trajectory-to-Skill Composition

Math considers trajectories from all 25 ES generations, keeps at most one failed trajectory for each of the 400 training problems, and excludes successful traces from skill distillation. A single post-hoc Trace2Skill pass uses GPT-5.4-nano, and temperature 1. We run each model with three seeds to distill the skill (base model, Agentic RL model, Agentic ESOpt model) and select the best result to report. Final evaluation reconstructs the same No Skill ES updates and injects the distilled skill only into the system context; it performs no further parameter optimization.

E.3 DocVQA

E.3.1 ReAct Prompt

DocVQA is also an agentic ReAct setting. The task exposes the document image path, and the agent may use command-line or Python tools to inspect the image, crop regions, or run OCR before submitting a short answer. The evaluator scores the final submitted answer with ANLS.

Prompt for DocVQA ReAct

Answer the document visual question. Inspect the referenced document image carefully. You can use command-line and Python tools over the image file.

[Optional skill instructions are inserted here.]

Action format:

Action:

```
{
  "name": "bash",
  "arguments": {"command": "<shell command>"}
}
```

After each Action, the environment returns an Observation. Repeat Thought/Action/Observation as needed. When finished, output only the short final answer.

Image path: data\trace2skill\docvqa\images\49153.png

Question: What is the 'actual' value per 1000, during the year 1975?

Thought: I need inspect the document chart and locate the 1975 value for the series labeled "actual".

Action:

```
{
  "name": "bash",
  "arguments": {"command": "python -c \"from PIL import Image; p=r'data\\trace2skill\\docvqa\\images\\49153.png'; img=Image.open(p); print(img.size)\""}
}
```

Observation: (1695, 2200)

Thought: I should zoom into the chart area around the 1975 tick and read the actual series label/value before answering.

Action:

```
{
  "name": "bash",
  "arguments": {"command": "python -c \"from PIL import Image; p=r'data\\trace2skill\\docvqa\\images\\49153.png'; img=Image.open(p); crop=img.crop((250,650,1450,1350)); crop.save('docvqa_49153_chart_crop.png'); print('saved docvqa_49153_chart_crop.png')\""}
}
```

Observation: saved docvqa_49153_chart_crop.png

Thought: The chart crop shows the 1975 actual value as 0.28 per 1000.
Final answer: 0.28

E.3.2 Trajectory-to-Skill Composition

DocVQA selects the final 50 task occurrences and keeps at most one failed and one successful trace per occurrence; when one outcome type is unavailable, only the observed type is retained. A single post-hoc Trace2Skill pass uses GPT-5.4-nano and temperature 1. We run each model with three seeds to distill the skill (base model, Agentic RL model, Agentic ESOpt model) and select the best result to report. Final evaluation reconstructs the same No Skill ES updates and injects the distilled skill only into the system context; it performs no further parameter optimization.

E.4 WebArena-Lite

E.4.1 Environment Prompt

WebArena-Lite uses the WebArena id-based accessibility-tree prompt. The full observation can be long, so the box shows the stable prompt structure and one real WebArena-Lite task objective.

Prompt for WebArena-Lite

You are an autonomous intelligent agent tasked with navigating a web browser. You will be given web-based tasks. The information includes the user's objective, the current accessibility tree, the current page address, open tabs, and the previous action.

Available actions include:

```
click [id]
type [id] [content] [press_enter_after=0|1]
hover [id]
press [key_comb]
scroll [down|up]
goto [url]
go_back
go_forward
stop [answer]
```

OBSERVATION:

[accessibility-tree nodes with element IDs]

CURRENT PAGE: [administration interface]

OBJECTIVE: What is the top-1 best-selling product in 2022

PREVIOUS ACTION: None

The response should reason briefly and end with:

In summary, the next action I will perform is <action>

E.4.2 Trace2Skill Policy Context

The following box presents the WebArena skill used by the Trace2Skill-conditioned agents.

Trace2Skill WebArena Skill

WebArena Skill

WebArena workflow

- Use only visible page evidence and current WebRL ids; re-resolve elements after any navigation, refresh, rerender, or layout change.
- Start from the most direct visible path. Prefer search, filters, menus, or built-in navigation before long scrolling or guessing controls.
- Re-read the current visible page after every click, selection, text entry, scroll, or navigation step. If the page did not visibly change, do not repeat the same action; choose a different visible control or route.
- Before typing, verify the field label, current value, and input type. Clear or select all existing text when overwriting.
- For dropdowns, pickers, autocomplete fields, and multi-selects, inspect the current control state first, choose the visible option that matches the task, and confirm the selected value is shown afterward.
- Treat search results, snippets, counts, and summary links as discovery aids only. Do not conclude completion until the requested state is visibly shown on the relevant page or detail view.
- If search or filter results are empty, ambiguous, or stuck, adjust the query, broaden or narrow the filter, or switch to a different grounded view instead of repeating near-identical searches.

- Use visible filters , chips , row counts , and result scopes as the source of truth for the current view .

Lists , pagination , and completeness

- For tasks that ask for all matching items , exhaust pagination , scrolling , and expansion until no more entries remain and the requested cardinality is confirmed .
- Use page controls and visible range indicators to move stepwise through the relevant span ; do not rely on a single viewport or old row positions .
- Keep a running tally or deduplicated set while scanning across pages or scroll regions when completeness matters .
- For count , total , or range questions , verify every contributing visible row or bucket directly and compute from the source values .
- For most , least , highest , or lowest questions , confirm the global extreme across the full relevant result set or a trustworthy aggregate before selecting the target .

Detail pages , reviews , and extraction

- Open the item detail page before answering any status , review , comment , or explanation question ; confirm the title and key state from the header , badge , or metadata rather than the list row .
- For review or thread-based questions , use explicit review , comment , or reply text from the dedicated discussion view ; do not infer from ratings , snippets , or summaries .
- Treat clipped snippets , ellipses , and partial text as incomplete evidence ; expand or open the page that shows the full content .
- When asked to extract text , copy the source text verbatim and keep only the exact requested span .
- If a likely match appears , verify its visible details on the detail page before answering or editing .

Forms , edits , and persistence

- Locate an explicit writable composer , dialog , or editor before typing ; do not type into read-only previews or summaries .
- Fill every required field explicitly and use the page's own save , apply , update , or submit control .
- After saving or submitting , verify the persisted record or changed state is visible on the destination page ; do not rely on a generic toast alone .
- If validation , required-field , login , modal , or blank/stale-state interruptions appear , resolve the issue or back out to the last stable page and continue with a different path .
- When editing a prefilled field , clear or select all first so the new value replaces the old one cleanly .
- For validated inputs , enter a format the field accepts and confirm success through a cleared error , visible confirmation , or reread saved value .
- For create-then-configure tasks , finish and confirm the created resource before moving to follow-up settings such as members , permissions , or access controls .
- Prefer reusing an existing item when it satisfies the task ; avoid creating duplicates unless creation is explicitly required .

Reports , maps , and directions

- For report or analytics tasks , use the site's dedicated report or analytics page first ; set the requested filters before generating the report .
- Match the report subview or page type to the question , and read the rendered results directly rather than the filter form or summary widgets .
- For place or location tasks , use the site search or map view to locate the anchor place first , then ground the answer in visible map context or a detailed result panel .
- Use directions mode for travel-time or reachability tasks , keep it separate from general search , and fill both endpoints with concrete locations .
- Verify the route summary , travel time , or distance from the route output , not from surrounding map UI .
- If a destination search fails or resolves incorrectly , refine the query or switch to a broader grounded place label instead of repeating the same guess .

Dead ends and login barriers

- If navigation lands on a login , cookie , account , or other blocking screen , treat it as a dead end : backtrack once to the last useful page and continue with a different route .
- Check for an existing signed-in session , account menu , or visible credential source before attempting login .
- Do not invent or brute-force credentials ; use only credentials provided by the task or visible page evidence .
- If a login attempt fails , read the error message and switch strategy instead of repeating similar guesses .

Completion rule

- Once the requested state , output , or persisted change is visibly confirmed , stop immediately and base the final answer only on page-visible evidence .
- Do not add extra verification after the completion condition is already satisfied .

E.4.3 Agentic ESOpt + Trace2Skill Policy Context

The following box presents the WebArena skill distilled from Agentic ESOpt trajectories and used for the combined result in [Table 4](#). It is reported separately from the Trace2Skill-only skill above because the two conditions use different trajectory sources.

```
# WebArena Skill

### Core workflow
- Use the most specific visible page or view that can directly show the requested signal; prefer list,
  detail, report, or management pages over inference.
- Use only the current visible page and DOM evidence after each navigation or state change; never
  reuse stale element ids.
- Before acting, re-scan the visible controls and match the requested verb to the exact UI control. Do
  not substitute a nearby action for the one the task asks for.
- Clear stale or conflicting filters before applying a new search or constraint, and confirm the
  active filters and sort order before extracting anything.
- For search-first tasks, use the page's actual search control or Enter, then confirm the page is
  truly showing search results before selecting anything.
- After each click, search, filter, or sort, verify that the page visibly changed. If the same action
  leaves the page unchanged or produces an error, stop repeating it and switch to a different
  control or path.
- For list-based tasks, do not stop at a preview row. Inspect enough pages or candidates to cover the
  full scope, and use pagination or page-jump controls when available.
- For plural or broad tasks, verify each requested item explicitly against the task criterion before
  exiting.
- Open the exact matching detail view before using a record's date, time, status, identifier, or other
  exact field as the answer.
- For form, edit, or dialog workflows, type only into writable controls, fill all required fields,
  confirm dropdown selections visibly changed, and use the explicit save, submit, or confirm
  control.
- After any save, submit, or other critical click, look for explicit visible success evidence and
  inspect a stable destination or reopened view once to confirm the persisted state matches the
  target.
- Do not trust transient banners, toasts, or typed text without visible page evidence that the change
  persisted.
- For discussion, comment, reply, or notification tasks, use the visible composer/editor and the
  explicit submit control from the record-level path.
- If an unexpected but related page opens, use back navigation or breadcrumbs to return to the source
  list or item and continue from there.
- If a click, submit, or navigation action has no visible effect, do not repeat it blindly; recover a
  visible state, locate the correct control, and try a different path.
- If the page becomes blank, stale, or unexpected, stop using old element ids and recover a visible
  state first by going back, reloading, or navigating to a known page.
- For toggle-style tasks, click the actual state-changing control, not a count, auxiliary link, or
  profile-like detour.
- For saved-item or wishlist-style tasks, confirm the item appears in the persistent saved view, not
  just in a transient confirmation message.
- Track every requested target explicitly in multi-item tasks and verify each one in the final visible
  state before exiting.
- Stop immediately once the requested page state, saved change, or answer is visibly confirmed; do not
  add extra exploration or verification after completion.
```

E.5 Automatic Heuristic Design

E.5.1 TSP-Construct Prompt

For AHD, the agent writes a heuristic function instead of a direct task solution. The following box shows the TSP constructive setting and representative EoH operator prompts. The concrete parent algorithms and code blocks are inserted at the placeholders for crossover or mutation operators.

Prompt for AHD TSP-Construct

```
Task:
Given a set of nodes with their coordinates, find the shortest route that visits each node once and
returns to the starting node. The task can be solved step-by-step by starting from the current
node and iteratively choosing the next node. Help me design a novel algorithm that is different
from algorithms in literature to select the next node in each step.

Function:
def select_next_node(current_node, destination_node, unvisited_nodes, distance_matrix):
    return next_node

Input/output information:
'current_node', 'destination_node', 'next_node', and 'unvisited_nodes' are node IDs. 'distance_matrix'
is the distance matrix of nodes. All are Numpy arrays.

EoH operator prompts:
[i1] First, describe your new algorithm and main steps in one sentence. The description must be inside
a brace. Next, implement it in Python as select_next_node(...). Do not give additional
```

```

explanations.
[e1] Given several existing algorithms and their code, create a new algorithm that has a totally
different form from the given ones.
[e2] Given several existing algorithms and their code, identify their common backbone idea, then
create a new algorithm with a different form that is motivated by that backbone.
[m1] Given one existing algorithm and its code, create a modified version with a different form.
[m2] Given one existing algorithm and its code, identify the main algorithm parameters and create a
new algorithm with different parameter settings of the score function.

```

F Licensing and External Components

F.1 Licensing Scope

The code introduced by this study is released under the MIT License. External code, benchmark assets, datasets, model checkpoints, and API services remain governed by their respective upstream licenses or terms. The table below is intentionally limited to repositories whose code or task data are directly invoked, vendored, or copied by the released workflow. Repositories consulted only as references, and baseline repositories whose implementations were not executed, are excluded.

The experiments additionally use Llama and Qwen checkpoints, API-hosted models, DAPO/AIME Math data, and DocVQA data under the terms of their respective authors and providers. Nothing in this release alters or supersedes those terms.

F.2 Upstream Repositories Used by the Released Workflow

Table 21 A summary of licenses and external resources used in this work.

Resources	Type	License	URL
Qwen3.5-4B	Base LLM	Apache-2.0 License	https://huggingface.co/Qwen/Qwen3.5-4B
Qwen3.5-9B	Base LLM	Apache-2.0 License	https://huggingface.co/Qwen/Qwen3.5-9B
Qwen3.5-27B	Base LLM	Apache-2.0 License	https://huggingface.co/Qwen/Qwen3.5-27B
LLaMA-3.1-8B-Instruct	Base LLM	Llama 3.1 Community License	https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct
verl	RL framework	Apache-2.0 License	https://github.com/volcengine/verl
EoH	AHD framework	MIT License	https://github.com/FeiLiu36/EoH
Trace2Skill	framework	Apache-2.0 License	https://github.com/Qwen-Applications/Trace2Skill
VisualAgentBench	Web-agent runtime	Apache-2.0 License	https://github.com/THUDM/VisualAgentBench
VisualWebArena	Web-agent environment	MIT License	https://github.com/web-arena-x/visualwebarena
AHD Problems	Dataset / Benchmark	MIT License	https://github.com/zz1358m/MCTS-AHD-master
DAPO-Math-17k	Dataset	Apache-2.0 License	https://huggingface.co/datasets/BytedTsinghua-SIA/DAPO-Math-17k
AIME 2026	Dataset	Apache-2.0 License	https://huggingface.co/datasets/math-ai/aime26
DocVQA	Dataset	Challenge Terms	https://site.docvqa.org/datasets
WebArena(-Lite)	Dataset / Benchmark	Research use only	https://github.com/liushiliushi/JitRL